PYTHON FOUNDATIONS

Python Foundations and Core Data Structures

A detailed, practice-first handbook for rebuilding Python knowledge and becoming a confident coder

16	100	Python 3
guided chapters	coding exercises	modern syntax

PART 1 OF THE PYTHON-TO-DATA-ANALYTICS PATH

Learn by reading, predicting, coding, testing, and explaining.

How to use this handbook

This book is designed for a candidate who has seen Python before but cannot recall it reliably. The objective is not passive familiarity. The objective is to rebuild a precise mental model, write correct code without copying, and choose data structures intentionally.

THE FIVE-STEP LEARNING LOOP

- Read the concept and restate it in your own words.
- Predict the output of each example before running it.
- Type the code yourself. Do not copy and paste during practice.
- Complete the nearby exercise before looking at the solution appendix.
- Explain the time complexity, edge cases, and data-structure choice aloud.

Recommended study setup

- Use a recent Python 3 interpreter. The examples avoid third-party packages.
- Keep one folder per chapter and one file per exercise.
- Run code frequently. Small feedback loops are more effective than reading many pages at once.
- Create a mistakes.md file. Record every bug, why it happened, and how you fixed it.
- After each chapter, solve at least one exercise again without viewing your earlier code.

Suggested practice folder

```
python_foundations/
    chapter_01/
        examples.py
        ex_001.py
        ex_002.py
    chapter_02/
        examples.py
        ex_005.py
    mistakes.md
```

How exercises are graded

Level	What it tests	Target behavior
Recall	Syntax and direct application	Finish without notes after one revision
Apply	Combine two or three ideas	Write, test, and explain edge cases
Challenge	Choose a structure and design the algorithm	State complexity and justify the design

DO NOT SKIP TRACING

For loops, nested structures, and mutable objects, write the state after each important line. Tracing builds the mental execution model that prevents random debugging.

Curriculum map

Part	Chapters	Capability
I. Foundations	1-7	Run Python, reason about values, write decisions, loops, and reusable functions
II. Core structures	8-14	Use lists, tuples, sets, dictionaries, nested records, comprehensions, and collections
III. Problem solving	15-16	Choose structures by complexity and solve integrated analytics-flavored problems
Appendices	A-D	Reference solutions, cheat sheets, glossary, and final readiness assessment

Six-week completion plan

Week	Primary chapters	Evidence of mastery
1	1-3	Write small scripts, convert types safely, and explain name-to-object binding
2	4-6	Clean text, branch correctly, and solve loop problems without accidental infinite loops
3	7-9	Design functions, manipulate lists, and use tuples and unpacking
4	10-12	Deduplicate with sets, aggregate with dictionaries, and traverse nested records
5	13-15	Write comprehensions, use collections, and justify complexity
6	16 + review	Complete integrated problems under time limits and explain every decision

Diagnostic checkpoint

Before starting, try these tasks without searching. The result tells you where to slow down:

1. Read five integers from a list and return the largest without calling max().
2. Count how many times each word occurs in a sentence.
3. Remove duplicate customer IDs while preserving their first-seen order.
4. Explain the difference between == and is.
5. State the average lookup cost for a list and a dictionary.

> **INTERPRETATION**
>
> Zero or one confident answer: begin at Chapter 1. Two or three: skim examples but complete every exercise. Four or five: move faster through Part I, but do not skip the mutability, hashability, and complexity sections.

PART 1

Python Foundations

Execution, values, decisions, loops, and reusable functions.

Chapters 1-7

CHAPTER 01

Running Python and Reading Programs

Build a reliable edit-run-observe-debug loop before learning more syntax.

LEARNING OUTCOMES

- Distinguish the interpreter, interactive shell, script, and source file.
- Read Python statements in execution order and respect indentation.
- Use print(), input(), comments, and simple error messages deliberately.
- Classify syntax errors, runtime errors, and logic errors.

1.1 What Python actually does

A Python program is text stored in a source file, usually with the .py extension. The Python interpreter reads that source, compiles it to an internal bytecode form, and executes the bytecode in a Python virtual machine. You do not normally manage those internal steps, but knowing that execution is sequential helps you trace code.

MENTAL MODEL

Unless control flow changes the path, Python executes statements from top to bottom. A name must be created before a later statement can use it.

A complete four-statement script

```
report_name = "Weekly Sales"
row_count = 128

print(report_name)
print("Rows processed:", row_count)
```

Output

```
Weekly Sales
Rows processed: 128
```

1.2 Interactive shell versus script

Environment	Best use	Limitation
Interactive shell / REPL	Test one expression, inspect a value, explore a method	Work is easy to lose and hard to review
.py script	Repeatable programs, exercises, automation, tests	Requires saving and rerunning the file
Notebook	Exploration, narrative analysis, visual output	Cells can run out of order and hide state problems

During this book, use scripts for exercises and the interactive shell for quick experiments. A script should run from a clean start. If code works only because a notebook cell was run earlier, the program is not reproducible.

1.3 Statements, expressions, and indentation

An expression produces a value. A statement performs an action such as assigning a name, calling a function, or changing control flow. The distinction is useful when reading errors and when deciding what can appear on the right side of an assignment.

Expressions inside statements

```
2 + 3                # expression: produces 5
subtotal = 2 + 3     # assignment statement
print(subtotal)      # function-call statement
```

Indentation is part of Python syntax. A colon introduces a block, and every statement in that block must be indented consistently. Four spaces is the standard indentation level.

Indentation defines the block

```
is_valid = True

if is_valid:
    print("Record accepted")
    print("Continue processing")

print("Validation finished")
```

> **COMMON TRAP**
>
> Tabs and spaces may look identical in an editor. Configure the editor to insert four spaces and display invisible whitespace when debugging indentation problems.

EX-001 | First reproducible script Recall - 5 min

Create a script that stores your name, target role, and weekly study hours in variables. Print a three-line learning profile. Run the file twice and confirm that the output is identical.

Constraints: Use three variables; Use at least two print() calls

Hint: *Start with strings for text and an integer for hours.*

1.4 Output with print()

print() converts its arguments to text and writes them to standard output. Multiple arguments are separated by a space by default. The sep parameter changes the separator, and end changes what is written after the final argument.

Useful print() options

```
region = "South"
orders = 42

print(region, orders)                    # South 42
print(region, orders, sep=" | ")        # South | 42
print("Loading", end="...")
print("done")                            # Loading...done
```

EX-002 | Console receipt Apply - 10 min

Write a script that prints a small receipt with a title, item name, quantity, unit price, and subtotal. Use sep or formatted strings so the result is readable.

Input / output: For item "Notebook", quantity 3, unit price 45.50, print subtotal 136.50.

Constraints: Store all source values in variables; Do not type the subtotal directly

Hint: *Compute subtotal = quantity * unit_price.*

1.5 Input is text until converted

input() pauses the program, displays a prompt, and returns exactly what the user typed as a string. Even digits such as 25 arrive as the string "25". Numeric work therefore requires explicit conversion.

Convert user input before arithmetic

```
age_text = input("Enter age: ")
age = int(age_text)
next_year = age + 1
print("Next year you will be", next_year)
```

DEBUGGING QUESTION

When an arithmetic expression fails, inspect both the value and its type. print(repr(value), type(value)) reveals hidden spaces and string values.

EX-003 | Minutes to seconds Recall - 7 min

Ask the user for a whole number of minutes, convert the input, and print the equivalent number of seconds.

Input / output: Input 12 should produce 720 seconds.

Constraints: Use input(); Convert with int(); Do not hard-code 60 * 12

1.6 Three families of errors

Error family	Meaning	Example
Syntax error	Python cannot parse the program	Missing colon or unmatched parenthesis
Runtime error	Valid syntax fails while executing	int("ten") or division by zero
Logic error	Program runs but gives the wrong result	Using + instead of * for subtotal

Read a traceback from the final line upward. The final line names the exception and usually explains the immediate problem. The file and line references above it show where the failing call occurred.

Valid code with a logic problem

```
quantity = 3
price = "45.50"
subtotal = quantity * price
print(subtotal)
```

Output

```
45.5045.5045.50
```

Multiplying a string by an integer repeats the string. Python followed its rules correctly; the programmer used the wrong type. Logic errors require reasoning, not only reading an exception.

EX-004 | Repair a broken invoice Apply - 8 min

Fix the program below so it prints a numeric total of 250.00. The original program is: price = "125.00"; quantity = 2; total = price * quantity.

Constraints: Keep the original string input; Convert before multiplication; Format two decimal places

Chapter summary

KEY IDEAS

- Python normally executes statements from top to bottom.
- Indentation defines blocks and must be consistent.
- input() returns a string; convert it before numeric work.
- A program can be syntactically valid and still be logically wrong.

COMMON MISTAKES

- Running code only in an old notebook state instead of from a clean start.
- Ignoring the final line of a traceback.
- Assuming user input is already numeric.
- Changing several lines at once while debugging.

PRACTICE RULE: After every small change, save, rerun, and inspect the exact output. Build confidence through fast feedback.

CHAPTER 02

Names, Objects, Types, and Memory

Replace the idea of "variables as boxes" with Python's name-to-object model.

LEARNING OUTCOMES

- Explain that assignment binds a name to an object.
- Use type(), id(), ==, and is correctly.
- Predict aliasing behavior for mutable objects.
- Create shallow copies when independent top-level containers are required.

2.1 Assignment binds names

In Python, a variable is best understood as a name bound to an object. The object has a type and a value. The name does not permanently own one type; a later assignment can bind the same name to a different object.

```
value = 10
print(type(value))

value = "ten"
print(type(value))
```

Output

```
<class 'int'>
<class 'str'>
```

PROFESSIONAL HABIT

Although Python permits rebinding a name to unrelated types, stable meaning makes code easier to review. Do not reuse total for a string message after using it as a number.

2.2 Identity, equality, and type

Tool	Question answered	Correct use
type(x)	What class created this object?	Inspection, validation, debugging
x == y	Do these values compare equal?	Normal value comparison
x is y	Are these references the same object?	Singleton checks such as x is None
id(x)	What identity number does this object have now?	Learning and low-level debugging

```
a = [10, 20]
b = [10, 20]
c = a

print(a == b)   # same contents
print(a is b)   # different list objects
print(a is c)   # same list object
```

Output

```
True
False
True
```

EX-005 | Safe swap Recall - 5 min

Store two region codes in names first and second. Swap them without using a temporary third name, then print the result.

Input / output: Before: first="AP", second="TS". After: first="TS", second="AP".

2.3 Mutable and immutable objects

An immutable object cannot be changed after creation. An operation that appears to modify it creates a new object and rebinds a name. A mutable object can change while retaining its identity.

Usually immutable	Usually mutable
int, float, bool, str, tuple, frozenset, bytes	list, set, dict, bytearray

Rebinding versus in-place mutation

```
text = "north"
old_id = id(text)
text = text.upper()
print(old_id == id(text))  # usually False: new string

values = [1, 2]
old_id = id(values)
values.append(3)
print(old_id == id(values))  # True: list changed in place
```

EX-006 | Trace an alias Apply - 8 min

Create a list called original with values [5, 10]. Bind alias = original, append 15 through alias, and print both names plus both identity values. Write one sentence explaining the result.

Hint: *Both names refer to the same list object.*

2.4 Copying containers

A shallow copy creates a new outer container but keeps references to the same nested objects. For a flat list, list.copy() is usually enough. For nested structures, decide whether shared inner objects are safe.

Independent top-level lists

```
source = [10, 20, 30]
independent = source.copy()
independent.append(40)

print(source)       # [10, 20, 30]
print(independent)  # [10, 20, 30, 40]
```

A shallow copy still shares nested lists

```
source = [[1, 2], [3, 4]]
shallow = source.copy()
shallow[0].append(99)

print(source)   # [[1, 2, 99], [3, 4]]
print(shallow)  # [[1, 2, 99], [3, 4]]
```

EX-007 | Independent scenario list Apply - 8 min

Start with baseline = [100, 120, 90]. Create a separate scenario list, increase its second value to 150, and prove that baseline remains unchanged.

Constraints: Use a shallow copy method; Do not recreate the list literal manually

EX-008 | Equality and identity report Apply - 10 min

Create two equal but separate dictionaries and a third name that aliases the first. Print a clear report showing == and is results for each pair.

2.5 None represents absence

None is a singleton object used to represent the absence of a value. Compare it with is None or is not None. Do not confuse None with 0, False, an empty string, or an empty collection; those are real values with different meanings.

Identity comparison is correct for None

```
manager = None

if manager is None:
    print("Manager not assigned")
```

EX-009 | Object inspection helper Challenge - 12 min

Write a function named inspect_value(value) that prints repr(value), type(value).__name__, id(value), and whether value is None. Call it with 0, "", None, and an empty list.

Hint: *The function chapter comes later; use the shown def syntax as a preview.*

Chapter summary

KEY IDEAS

- Assignment binds names to objects; it does not automatically copy objects.
- Use == for value comparison and is primarily for identity checks such as is None.
- Mutation changes an existing object; rebinding points a name to another object.
- A shallow copy duplicates only the outer container.

COMMON MISTAKES

- Using is to compare strings or numbers.
- Assuming alias = original makes a copy.
- Treating None, 0, False, and empty collections as the same business state.
- Mutating a shared list without understanding who else references it.

PRACTICE RULE: Before modifying a collection, ask: is this name the only reference, or could another part of the program observe the mutation?

CHAPTER 03

Numbers, Booleans, Operators, and Conversion

Perform calculations precisely and make assumptions visible.

LEARNING OUTCOMES

- Use integer, floating-point, Boolean, and complex values appropriately.
- Apply arithmetic, comparison, logical, and assignment operators.
- Use floor division and modulo for quotient-remainder problems.
- Convert text safely and compare floating-point results with tolerance.

3.1 Numeric types

Type	Example	Use
int	42, -7, 0	Counts, indexes, exact whole-number arithmetic
float	3.14, -0.5, 1e6	Measurements and approximate decimal arithmetic
bool	True, False	Conditions; bool is a specialized integer type
complex	2 + 3j	Scientific and engineering calculations

Python integers grow as large as available memory permits. Floating-point values use a finite binary representation, so many decimal fractions cannot be stored exactly. Financial systems often use Decimal or integer minor units, but the built-in float remains common for general analytics.

3.2 Arithmetic and precedence

Operator	Meaning	Example result
+	addition	7 + 3 -> 10
-	subtraction	7 - 3 -> 4
*****	multiplication	7 * 3 -> 21
/	true division	7 / 3 -> 2.333...
//	floor division	7 // 3 -> 2
%	remainder	7 % 3 -> 1
******	exponentiation	2 ** 3 -> 8

Parentheses communicate intent and protect calculations from misunderstood precedence. Exponentiation binds more tightly than multiplication, which binds more tightly than addition. Comparisons are evaluated after arithmetic, and not, and, or have progressively lower logical precedence.

```
revenue = 250_000
cost = 175_000
margin_pct = (revenue - cost) / revenue * 100
print(f"Margin: {margin_pct:.1f}%")
```

Output

```
Margin: 30.0%
```

EX-010 | Temperature converter Recall - 6 min

Convert a Celsius value to Fahrenheit using F = C * 9 / 5 + 32. Print one decimal place.

Input / output: 37 C should produce 98.6 F.

3.3 Quotient and remainder patterns

Floor division and modulo are often used together. For a non-negative total, total // unit gives the number of complete units and total % unit gives what remains. This pattern converts seconds to hours and minutes, pages to batches, or records to full chunks.

```
total_seconds = 9_125
hours, remainder = divmod(total_seconds, 3_600)
minutes, seconds = divmod(remainder, 60)
print(hours, minutes, seconds)
```

Output

```
2 32 5
```

EX-011 | Seconds to clock time Apply - 12 min

Convert a non-negative total number of seconds into hours, minutes, and seconds. Format it as HH:MM:SS.

Input / output: 9125 -> 02:32:05.

Constraints: Use // and % or divmod(); Pad each field to two digits

3.4 Comparisons and Boolean logic

Expression	Meaning
a == b	equal values
a != b	different values
a < b, a <= b	less than, less than or equal
a > b, a >= b	greater than, greater than or equal
x and y	both conditions must be truthy
x or y	at least one condition must be truthy
not x	negate the truth value

```
amount = 18_000
is_verified = True
within_limit = amount <= 20_000
approved = is_verified and within_limit
print(approved)
```

Output

```
True
```

Python supports chained comparisons. The expression 0 <= score <= 100 is clearer than 0 <= score and score <= 100, and the middle expression is evaluated only once.

EX-012 | Eligibility rule Apply - 8 min

A transaction is eligible when the amount is between 500 and 20,000 inclusive, the account is active, and the transaction is not blocked. Compute one Boolean named eligible.

EX-013 | Divisibility classifier Apply - 12 min

Given an integer, print whether it is divisible by 2, by 3, by both, or by neither. First compute reusable Boolean expressions.

3.5 Conversion and validation boundaries

int(), float(), str(), and bool() create values of the requested type when conversion is meaningful. Conversion is not the same as validation. int("12") succeeds, while int("12.5") and int("twelve") fail. Production code normally catches conversion errors at the input boundary.

```
raw = " 125.50 "
amount = float(raw.strip())
print(amount)
```

Output

```
125.5
```

BOOLEAN CONVERSION WARNING

bool("False") is True because every non-empty string is truthy. Parse textual flags explicitly instead of calling bool() on user text.

EX-014 | Invoice total Apply - 15 min

Calculate subtotal, discount amount, discounted subtotal, tax, and final total from quantity, unit price, discount percent, and tax percent. Display all money values with two decimal places.

Input / output: quantity=4, price=250, discount=10%, tax=18% -> total 1062.00.

3.6 Floating-point comparison

```
print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)
```

Output

```
0.30000000000000004
False
```

The result is not a Python bug. It follows binary floating-point representation. Compare measured or calculated floats with math.isclose(), choosing a relative or absolute tolerance appropriate to the domain.

```
from math import isclose

actual = 0.1 + 0.2
expected = 0.3
print(isclose(actual, expected, rel_tol=1e-9, abs_tol=1e-12))
```

Output

```
True
```

EX-015 | Tolerance-based reconciliation Apply - 8 min

Two systems report totals 125000.10 and 125000.11. Consider them reconciled when their absolute difference is at most 0.02. Print the difference and reconciliation result.

3.7 Safe ratios and analytics metrics

Ratios require a denominator policy. A denominator of zero may mean no opportunity, missing data, or a data-quality defect. Do not silently return an arbitrary percentage without defining the business meaning.

```
conversions = 45
visits = 0
conversion_rate = None if visits == 0 else conversions / visits
print(conversion_rate)
```

Output

```
None
```

EX-016 | KPI calculator Challenge - 12 min

Calculate gross margin percent and average order value. Return None for a metric whose denominator is zero. Use revenue=500000, cost=350000, order_count=0.

Constraints: Do not divide by zero; Keep unavailable values as None

Chapter summary

KEY IDEAS

- Use // and % together for quotient-remainder conversions.
- Parentheses make business formulas easier to audit.
- Boolean expressions should name the rule being implemented.
- Float comparison requires a domain-appropriate tolerance.
- Every ratio needs an explicit zero-denominator policy.

COMMON MISTAKES

- Assuming round() changes the stored value used later when only formatting was needed.
- Calling bool() on text such as "False".
- Comparing calculated floats with exact equality.
- Hiding several business steps inside one unreadable expression.

PRACTICE RULE: For every metric, write the formula, unit, denominator policy, and display precision before coding.

CHAPTER 04

Strings and Text Processing

Treat text as structured data: normalize deliberately, preserve meaning, and avoid accidental corruption.

LEARNING OUTCOMES

- Index and slice strings safely.
- Normalize case and surrounding whitespace without losing the raw source.
- Split structured text, join tokens, search, replace, and format output.
- Recognize that strings are immutable sequences of Unicode characters.

4.1 Strings are immutable sequences

A string is an ordered sequence of Unicode characters. Indexing retrieves one character and slicing retrieves a range. Indexes start at zero; negative indexes count from the end. Because strings are immutable, methods return new strings rather than modifying the original.

Indexing and slicing

```
text = "Python"
print(text[0])      # P
print(text[-1])     # n
print(text[1:4])    # yth
print(text[:3])     # Pyt
print(text[::2])    # Pto
print(text[::-1])   # nohtyP
```

SLICE SAFETY

A single out-of-range index raises IndexError, but an oversized slice is clipped safely. text[100] fails; text[:100] returns the available text.

EX-017 | Initials extractor Recall - 5 min

Given first_name="Ada" and last_name="Lovelace", create the initials "AL" using indexing and print them.

4.2 Normalization is a pipeline

Real text often contains accidental spaces, inconsistent case, and empty-looking values. A normalization pipeline should be explicit and repeatable. Preserve the raw value when auditability matters, then derive a clean value for matching or grouping.

```
raw_region = "  south INDIA  "
normalized_region = " ".join(raw_region.split()).casefold()

print(repr(raw_region))
print(repr(normalized_region))
```

Output

```
'  south INDIA  '
'south india'
```

Method	Purpose	Example
strip()	Remove whitespace from both ends	" AP ".strip() -> "AP"
lstrip() / rstrip()	Remove from one end	Useful for prefixes or line endings
lower()	Lowercase for display or simple matching	"South" -> "south"
casefold()	Aggressive caseless comparison	Preferred for international matching
title()	Title-case words	Useful for display, not identity
split()	Split on whitespace and collapse runs	"a b" -> ["a", "b"]

EX-018 | Customer-name normalizer Apply - 10 min

Normalize raw_name=" aNITA rAO " so the stored matching form is "anita rao" and the display form is "Anita Rao". Preserve the original value.

4.3 Searching and replacing

Operation	Behavior
sub in text	Membership test; returns True or False
text.find(sub)	Index of first occurrence or -1
text.index(sub)	Index of first occurrence or raises ValueError
text.count(sub)	Number of non-overlapping occurrences
text.startswith(prefix)	Prefix test
text.endswith(suffix)	Suffix test
text.replace(old, new)	New string with replacements

Use membership when only existence matters. Use find() when -1 is a convenient missing value. Use index() when absence should be exceptional. Replacement is literal; it does not understand words or patterns.

EX-019 | Email components Apply - 8 min

Given email="analyst@example.com", extract the local part and domain using find() or split(). Print both and whether the domain ends with ".com".

4.4 split(), partition(), and join()

split() returns a list of pieces. partition(separator) always returns exactly three strings: the text before the first separator, the separator itself, and the text after it. join() combines an iterable of strings with a chosen separator.

```
record = "TXN-1042|2026-11-03|1250.50|SUCCESS"
parts = record.split("|")
transaction_id, date_text, amount_text, status = parts

print(transaction_id)
print(" / ".join([date_text, status]))
```

Output

```
TXN-1042
2026-11-03 / SUCCESS
```

BOUNDARY RULE

Parsing is a boundary. Verify the number of fields and convert each field explicitly before trusting the record.

EX-020 | Transaction-line parser Apply - 12 min

Parse "TXN-9|South|2500.75|approved" into four names. Normalize status to uppercase and convert amount to float. Print a readable one-line summary.

EX-021 | File-name builder Apply - 10 min

Build the file name "south_sales_2026_11.csv" from region=" South ", report="Sales", year=2026, month=11. Normalize text to lowercase and replace spaces with underscores.

4.5 Character tests

Method	True when...
isalpha()	all characters are alphabetic and string is non-empty
isdigit()	all characters are digit characters
isdecimal()	all characters are decimal digits
isalnum()	all characters are alphabetic or numeric
isspace()	all characters are whitespace
islower() / isupper()	cased characters follow the requested case

These methods answer character questions; they do not validate a complete business identifier. A customer ID may require a prefix, a fixed length, and digits in specific positions.

EX-022 | Simple customer-ID validator Apply - 8 min

A valid ID has exactly 8 characters, begins with "CU", and the remaining 6 characters are decimal digits. Compute a Boolean is_valid for "CU004219".

4.6 Formatting output with f-strings

Formatted string literals place expressions inside braces. A format specification controls alignment, width, sign, decimal precision, thousands separators, percentages, and dates. Formatting changes the displayed string, not the original numeric value.

```
region = "South"
revenue = 1_245_678.9
margin = 0.2746

print(f"{region:<12} {revenue:>15,.2f} {margin:>8.1%}")
```

Output

```
South           1,245,678.90    27.5%
```

EX-023 | Fixed-width report row Apply - 12 min

Print a report row containing product, units, revenue, and margin. Use left alignment for the product, right alignment for numbers, a thousands separator, two decimals for revenue, and one decimal percentage for margin.

4.7 Escapes, raw strings, and representation

Escape sequences such as \n, \t, and \ backslash have special meaning. repr(text) returns an unambiguous representation that exposes escapes and surrounding quotes, making it valuable during cleaning and debugging. A raw string prefix r is useful for regular-expression patterns and many Windows-style paths, but a raw string cannot end with a single backslash.

```
message = "first line\nsecond line"
print(message)
print(repr(message))
```

Output

```
first line
second line
'first line\nsecond line'
```

EX-024 | Account-number masker Apply - 8 min

Given account="123456789012", keep only the final four characters visible and replace every earlier character with "*". The result should be "********9012".

Constraints: Use slicing; Do not hard-code eight asterisks

Chapter summary

KEY IDEAS

- Strings are immutable; methods return new strings.
- Preserve raw text when traceability matters and derive normalized fields.
- split() separates text, while join() combines strings.
- Use repr() to reveal whitespace and escape characters.
- Formatting controls presentation without changing source values.

COMMON MISTAKES

- Using title() or lower() as a universal identity rule without understanding the data.
- Assuming replace() understands word boundaries.
- Splitting without validating the number of fields.
- Using string slicing without considering short or empty input.

PRACTICE RULE: For every text-cleaning rule, keep an example of the raw input, the expected normalized value, and a reason for the transformation.

CHAPTER 05

Conditions and Decision Making

Translate business rules into readable, testable branches.

LEARNING OUTCOMES

- Use if, elif, and else to express mutually exclusive paths.
- Understand truthiness without collapsing distinct business states.
- Simplify nested logic with named conditions and guard clauses.
- Test boundaries systematically rather than with only a typical example.

5.1 The shape of a decision

Only the first true branch runs

```
amount = 12_500

if amount < 0:
    category = "invalid"
elif amount == 0:
    category = "zero"
elif amount <= 10_000:
    category = "standard"
else:
    category = "high"

print(category)
```

Output

```
high
```

An if-elif-else chain is evaluated from top to bottom. When a condition is true, its block runs and the remaining branches are skipped. Order therefore matters when ranges overlap. Put specific exceptions before broader rules.

EX-025 | Number sign classifier Recall - 6 min

Read or assign an integer and print exactly one of: positive, negative, or zero.

5.2 Truthiness

Conditions convert values to Boolean truth. None, False, numeric zero, and empty strings or collections are falsey. Most other objects are truthy. This is convenient for existence checks, but it can erase meaningful differences between missing, empty, and zero.

Choose truthiness only when the meanings can be combined

```
comment = ""

if not comment:
    print("No comment text")

score = 0
if score is None:
    print("Score missing")
else:
    print("Score recorded, including zero")
```

BUSINESS-STATE WARNING

if not value treats None, 0, False, "", [], {}, and set() alike. Use explicit comparisons when those states require different actions.

5.3 Range rules and boundary testing

Many defects occur at boundaries: exactly 0, exactly 60, exactly 100, one below a limit, or one above it. For every range rule, write tests for each boundary and adjacent values.

Descending thresholds prevent overlap

```
score = 82

if not 0 <= score <= 100:
    grade = "invalid"
elif score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F
```

EX-026 | Grade band Apply - 12 min

Implement the grade rule shown above as a complete script. Test -1, 0, 59, 60, 79, 80, 89, 90, 100, and 101.

Constraints: Reject values outside 0 to 100; Use one if-elif-else chain

5.4 Named conditions reduce cognitive load

Long Boolean expressions are easier to review when intermediate conditions are named after the rule. A good name explains why the condition matters, not merely which fields it compares.

Rule-oriented names

```
amount = 9_000
country = "IN"
kyc_complete = True
account_frozen = False

within_domestic_limit = country == "IN" and amount <= 10_000
identity_ok = kyc_complete
account_ok = not account_frozen
approved = within_domestic_limit and identity_ok and account_ok
```

EX-027 | Shipping-fee rule Apply - 15 min

Shipping is free when order value is at least 1,000 or the customer is a premium member. Otherwise it costs 80. However, delivery to a remote area adds 120 even when base shipping is free. Compute the final fee.

Input / output: order=800, premium=True, remote=True -> 120.

5.5 Guard clauses

Inside a function, a guard clause handles an invalid or exceptional case early and returns. This prevents deep nesting and lets the main path remain visually clear.

Invalid and empty cases exit before the main formula

```
def conversion_rate(conversions, visits):
    if visits < 0 or conversions < 0:
        raise ValueError("counts cannot be negative")
    if visits == 0:
        return None
    return conversions / visits
```

EX-028 | Record-quality status Apply - 10 min

Classify an amount as "missing" when it is None, "invalid" when it is negative, "zero" when it is 0, and "valid" otherwise. Preserve the distinction between all four states.

5.6 Conditional expressions

A conditional expression chooses one of two values: value_if_true if condition else value_if_false. Use it for a short assignment, not for multi-step business logic or nested decision trees.

```
status = "active"
label = "Eligible" if status == "active" else "Ineligible"
print(label)
```

Output

```
Eligible
```

EX-029 | Leap-year decision Challenge - 12 min

A year is a leap year when it is divisible by 400, or divisible by 4 but not by 100. Implement the rule and test 1900, 2000, 2024, and 2025.

5.7 Rule tables before code

When several flags interact, create a small decision table before coding. A table reveals missing combinations and conflicting rules. The code should then mirror the priority expressed by the table.

Verified	Amount > 50,000	Watchlist	Risk result
No	Any	Any	HIGH
Yes	Any	Yes	HIGH
Yes	Yes	No	MEDIUM
Yes	No	No	LOW

EX-030 | Risk classification Challenge - 15 min

Implement the decision table above. Test at least one example for every row and explain why rule order matters.

Chapter summary

KEY IDEAS

- An if-elif-else chain runs only the first true branch.
- Rule order matters when conditions overlap.
- Truthiness is useful only when falsey states have equivalent meaning.
- Named conditions and guard clauses make policy logic auditable.
- Boundary tests are mandatory for range-based decisions.

COMMON MISTAKES

- Placing a broad condition before a specific exception.
- Using if value when zero and missing have different meanings.
- Writing deeply nested logic instead of handling invalid cases early.
- Testing only one typical input.

PRACTICE RULE: For each decision, list the rule priority and boundary examples before writing the branch structure.

CHAPTER 06

Loops, Iteration, and State

Process repeated data while keeping state changes visible and controlled.

LEARNING OUTCOMES

- Use for and while loops for their appropriate purposes.
- Iterate with range(), enumerate(), and zip().
- Build totals, counts, searches, and filters with accumulator patterns.
- Use break and continue deliberately and avoid mutating a collection during traversal.

6.1 for loops consume iterables

A for loop requests one item at a time from an iterable. Strings, lists, tuples, dictionaries, sets, files, ranges, and generators are iterable. The loop variable is rebound for each item.

```
amounts = [125.0, 80.0, 210.0]
total = 0.0

for amount in amounts:
    total += amount

print(total)
```

Output

```
415.0
```

TRACE THE STATE

For accumulator loops, create a small table with iteration, current item, and accumulator after the update. This makes off-by-one and wrong-update errors visible.

EX-031 | Sum without sum() Recall - 6 min

Calculate the sum of [12, 5, 8, 20] using a for loop and an accumulator. Do not call sum().

6.2 range() generates integer sequences

Form	Values produced
range(stop)	0 through stop - 1
range(start, stop)	start through stop - 1
range(start, stop, step)	advance by step; stop remains exclusive

```
for index in range(5):
    print(index, end=" ")

for value in range(10, 4, -2):
    print(value, end=" ")
```

Output

```
0 1 2 3 4 10 8 6
```

EX-032 | Factorial Apply - 10 min

Calculate n! for a non-negative integer n using a loop. By definition, 0! is 1.

Input / output: n=5 -> 120.

6.3 enumerate() and zip()

enumerate(iterable, start=0) produces index-item pairs. zip() walks several iterables in parallel and stops when the shortest input ends. Use enumerate() instead of maintaining a manual counter and zip() instead of indexing parallel lists.

Indexed parallel iteration

```
regions = ["North", "South", "West"]
revenues = [120_000, 175_000, 98_000]

for rank, (region, revenue) in enumerate(
    zip(regions, revenues), start=1
):
    print(rank, region, revenue)
```

EX-033 | Revenue rows Apply - 12 min

Given products=["A", "B", "C"] and quantities=[3, 5, 2] with prices=[100, 80, 150], print a numbered row for each product and its revenue.

6.4 Accumulator patterns

Goal	Initial state	Update pattern
Total	0 or 0.0	total += value
Product	1	product *= value
Count	0	if condition: count += 1
Maximum	first item or None	replace when item is larger
Search	None / False	set result and break when found
Filter	empty list	append items that pass

EX-034 | Valid-value summary Apply - 12 min

For values=[120, -5, None, 0, 75, 210], count values that are not None and non-negative, and calculate their total. Treat zero as valid.

6.5 break, continue, and loop else

break exits the nearest loop immediately. continue skips the rest of the current iteration. A loop else block runs only when the loop finishes normally, not when break executes. Use these features sparingly; clear state variables are sometimes easier to understand.

Search with loop else

```
target = "TXN-103"
records = ["TXN-101", "TXN-102", "TXN-103"]

for record in records:
    if record == target:
        print("Found", record)
        break
else:
    print("Not found")
```

EX-035 | First threshold breach Apply - 10 min

Find the first value greater than 100 in [30, 75, 90, 140, 200]. Print the value and its zero-based index, then stop. Print "none" if no value breaches the threshold.

6.6 while loops are condition-driven

Use a while loop when the number of repetitions is not known in advance and execution depends on changing state: retries, interactive input, queue processing, or convergence. The loop must change something that can eventually make the condition false.

A bounded retry loop

```
attempts_left = 3
success = False

while attempts_left > 0 and not success:
    # Replace this with a real operation.
    success = attempts_left == 1
    attempts_left -= 1

print("Success:", success)
```

INFINITE-LOOP CHECKLIST

- Which variable controls the condition?
- Where does that variable change?
- Can every path reach the update?
- Is there a maximum-attempt or timeout guard?

EX-036 | Digit count Challenge - 12 min

Count the decimal digits in a non-negative integer using a while loop and integer division. Define 0 as having one digit.

Input / output: 50720 -> 5 digits.

6.7 Nested loops and cost

A nested loop may perform outer_count multiplied by inner_count iterations. Sometimes this is necessary, but it can become slow for large data. Before nesting, ask whether a set or dictionary can replace repeated searching. Those structures are introduced later.

Twelve coordinate pairs

```
rows = 3
columns = 4

for row in range(rows):
    for column in range(columns):
        print((row, column), end=" ")
    print()
```

EX-037 | Multiplication grid Apply - 12 min

Print a 5 by 5 multiplication grid using nested loops. Each row should contain five right-aligned values.

Chapter summary

KEY IDEAS

- for loops are ideal for consuming known iterables; while loops are condition-driven.
- range() excludes its stop value.
- enumerate() supplies indexes and zip() walks parallel iterables.
- Accumulators must start with the correct identity value.
- Nested loops multiply work and should trigger a data-structure review.

COMMON MISTAKES

- Manually managing an index when enumerate() is clearer.
- Forgetting that zip() stops at the shortest input.
- Changing a list while iterating over it.
- Writing an unbounded while loop without a state update.

PRACTICE RULE: Trace at least the first three iterations and the final iteration of every new loop before trusting it.

CHAPTER 07

Functions, Parameters, Return Values, and Scope

Turn repeated logic into small contracts that are easy to test and reuse.

LEARNING OUTCOMES

- Define functions with focused responsibilities and meaningful names.
- Distinguish parameters, arguments, return values, and printed output.
- Use positional, keyword, default, variadic, and keyword-only parameters responsibly.
- Understand local scope and avoid mutable default arguments.

7.1 A function is a contract

A useful function has an input contract, one focused responsibility, a clear return shape, and a documented failure policy. It should be possible to test the function without reading from input() or depending on unrelated global state.

```
def calculate_revenue(quantity, unit_price):
    # Return revenue for a non-negative quantity and price.
    if quantity < 0 or unit_price < 0:
        raise ValueError("quantity and price must be non-negative")
    return quantity * unit_price

revenue = calculate_revenue(5, 125.0)
print(revenue)
```

Output

```
625.0
```

RETURN IS NOT PRINT

print() displays a value for a human. return sends a value back to the caller so other code can store, test, transform, or combine it.

EX-038 | Name formatter Apply - 10 min

Write format_name(first, last) that strips extra whitespace, converts both parts to title case, and returns "Last, First". Do not print inside the function.

7.2 Parameters and arguments

Term	Meaning
Parameter	Name in the function definition
Argument	Value supplied by the caller
Positional argument	Matched by position
Keyword argument	Matched by parameter name
Default parameter	Used when the caller omits that argument

Term	Meaning
Keyword-only parameter	Must be passed using its name

The * makes later parameters keyword-only

```
def format_currency(amount, *, symbol="INR", decimals=2):
    return f"{symbol} {amount:,.{decimals}f}"

print(format_currency(1250.5, symbol="Rs", decimals=2))
```

EX-039 | BMI calculator Apply - 12 min

Write calculate_bmi(weight_kg, height_m) that returns weight / height squared. Reject non-positive inputs with ValueError. Round only when displaying the result.

7.3 Returning several values

A function can return several values separated by commas. Python packages them in a tuple, and the caller can unpack the tuple into names. Keep the order stable and document it. For many named outputs, a dictionary or data class may be clearer; those are introduced later.

```
def min_max(values):
    if not values:
        return None, None
    return min(values), max(values)

low, high = min_max([8, 3, 11, 4])
print(low, high)
```

Output

```
3 11
```

EX-040 | Safe percentage Apply - 10 min

Write safe_percentage(part, whole) that returns None when whole is zero and otherwise returns part / whole * 100. Reject negative inputs.

EX-041 | Manual numeric summary Challenge - 18 min

Write summarize(values) that returns a tuple containing count, total, minimum, maximum, and mean. For an empty list, return (0, 0, None, None, None). Do not call sum().

7.4 Default arguments and the mutable-default trap

Default expressions are evaluated once when the def statement runs, not every time the function is called. Using a mutable list or dictionary as a default can therefore share state across calls. Use None as a sentinel and create the new container inside the function.

```
def add_tag(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags

print(add_tag("new"))
print(add_tag("priority"))
```

Output

```
['new']
['priority']
```

EX-042 | Invoice function Challenge - 20 min

Write calculate_invoice(quantity, unit_price, *, discount_pct=0, tax_pct=0) that validates non-negative inputs and returns subtotal, discount, tax, and total as a tuple.

Constraints: discount_pct and tax_pct must be passed by name; Apply tax after discount

7.5 Scope and name resolution

Names assigned inside a function are local unless declared otherwise. Python looks for a name in local, enclosing, global, and built-in scopes. Prefer passing values in and returning results rather than modifying global variables. Explicit data flow makes tests and parallel execution safer.

Global configuration is passed explicitly

```
tax_rate = 0.18

def add_tax(amount, rate):
    total = amount * (1 + rate)
    return total

print(add_tax(1000, tax_rate))
```

EX-043 | Reusable ID validator Apply - 12 min

Convert the customer-ID rule from EX-022 into is_valid_customer_id(customer_id). It must return False for non-string input instead of raising an error.

7.6 *args and **kwargs

*args collects additional positional arguments into a tuple. **kwargs collects additional keyword arguments into a dictionary. They are valuable for adapters and decorators, but explicit parameters are clearer for ordinary business functions because they document the expected interface.

```
def average(*values):
    if not values:
        return None
    return sum(values) / len(values)

print(average(10, 20, 30))
```

Output

```
20.0
```

EX-044 | Configurable discount Challenge - 15 min

Write apply_discount(amount, *, percent=0, minimum_total=0). Apply the discount only when amount is at least minimum_total. Validate percent is between 0 and 100.

Chapter summary

KEY IDEAS

- Functions should receive data, perform one responsibility, and return a stable result.
- Printing and returning solve different problems.
- Keyword-only parameters make policy-like options explicit.
- Use None instead of a mutable default container.
- Prefer explicit parameters and return values over global mutation.

COMMON MISTAKES

- Printing a result inside a function that callers need to reuse.
- Returning inconsistent shapes on different paths.
- Using a mutable list or dictionary as a default argument.
- Accepting invalid input without a documented policy.

PRACTICE RULE: For every function, write one normal test, one boundary test, and one invalid-input test before considering it complete.

PART 2

Core Data Structures

Ordered sequences, uniqueness, mappings, records, and specialized collections.

Chapters 8-14

CHAPTER 08

Lists: Ordered, Mutable Collections

Store sequences, update them deliberately, and recognize when list operations become expensive.

LEARNING OUTCOMES

- Create, index, slice, update, copy, and traverse lists.
- Choose among append(), extend(), insert(), remove(), pop(), sort(), and sorted().
- Build list algorithms for filtering, searching, rotation, and merging.
- Explain the cost of indexing, membership, insertion, and deletion.

8.1 List fundamentals

A list is an ordered, mutable collection that can hold references to objects of any type. In analytics code, lists often represent a sequence of measurements, records awaiting processing, column names, or intermediate results. Homogeneous contents are usually easier to reason about even though Python does not require them.

```
values = [12, 8, 15, 3]
print(values[0])       # 12
print(values[-1])      # 3
print(values[1:3])     # [8, 15]

values[2] = 20
print(values)
```

Output

```
12
3
[8, 15]
[12, 8, 20, 3]
```

Core list operations

Method / operation	Meaning	Typical cost
values[i]	Read or replace one position	O(1)
append(x)	Add one item at the end	amortized O(1)
extend(items)	Add all items from an iterable	O(k)
insert(i, x)	Insert and shift later items	O(n)
pop()	Remove and return final item	O(1)
pop(i)	Remove position i and shift later items	O(n)
remove(x)	Remove first equal value	O(n)
x in values	Linear membership search	O(n)
len(values)	Number of items	O(1)

EX-045 | List editing drill Recall - 8 min

Start with regions=["North", "East", "West"]. Add "South" to the end, insert "Central" at index 2, replace "East" with "Northeast", and remove "West". Print the final list.

8.2 Mutation versus a new list

Operation	Mutates original?	Returns
values.append(x)	Yes	None
values.sort()	Yes	None
values.reverse()	Yes	None
sorted(values)	No	New sorted list
values + other	No	New combined list
values.copy()	No	New shallow list
values[start:stop]	No	New shallow list

In-place method versus built-in function

```
scores = [72, 95, 81]
result = scores.sort()
print(result)  # None
print(scores)  # [72, 81, 95]

scores = [72, 95, 81]
ordered = sorted(scores)
print(scores)   # original order retained
print(ordered)
```

EX-046 | Scenario isolation Apply - 12 min

Create a baseline list [120, 150, 90, 200]. Create an independent scenario, increase every scenario value by 10 percent using a loop, and prove baseline is unchanged.

8.3 Filtering and transformation

A common loop creates a result list, examines each source item, and appends only acceptable values. Do not remove items from the same list being iterated; shifting indexes can skip elements.

```
raw_values = [120, None, -5, 0, 80]
valid_values = []

for value in raw_values:
    if value is None or value < 0:
        continue
    valid_values.append(value)

print(valid_values)
```

Output

```
[120, 0, 80]
```

EX-047 | Clean amount list Challenge - 18 min

From raw=["120.5", "", " 80 ", "invalid", "0"], build a list of successfully converted float values. Use a loop and try/except. Skip empty and invalid entries.

8.4 Searching and duplicate handling

List membership is a linear scan. For a short list or a single search, that is often fine. Repeating membership checks while building a result can become quadratic. A set will later reduce lookup cost, but first understand the list-only pattern.

```
customer_ids = ["C1", "C2", "C1", "C3", "C2"]
unique_in_order = []

for customer_id in customer_ids:
    if customer_id not in unique_in_order:
        unique_in_order.append(customer_id)

print(unique_in_order)
```

Output

```
['C1', 'C2', 'C3']
```

EX-048 | Order-preserving deduplication Apply - 12 min

Remove duplicates from [4, 2, 4, 1, 2, 3, 1] while preserving first-seen order. Use only lists for this version and state the worst-case time complexity.

8.5 Sorting with keys

sorted() and list.sort() accept key, a function that extracts the comparison value, and reverse, a Boolean for descending order. Python sorting is stable, so equal-key items retain their earlier relative order.

Sort records by revenue

```
records = [
    ["North", 120_000],
    ["South", 175_000],
    ["West", 98_000],
]

ordered = sorted(records, key=lambda row: row[1], reverse=True)
print(ordered)
```

EX-049 | Top three distinct values Apply - 15 min

Return the three largest distinct values from [10, 40, 20, 40, 30, 10, 50]. Use list operations only in this exercise.

8.6 Slicing patterns

Common slices

```
values = [10, 20, 30, 40, 50]
print(values[:3])      # first three
print(values[-2:])     # final two
print(values[1::2])    # every second item from index 1
print(values[::-1])    # reversed copy
```

Slicing creates a shallow copy. A full slice values[:] is an alternative to values.copy(). For a large list, remember that slicing copies references and therefore uses time and memory proportional to the slice length.

EX-050 | Rotate right Challenge - 15 min

Rotate [1, 2, 3, 4, 5] right by k positions using slicing. Support k values larger than the list length.

Input / output: k=2 -> [4, 5, 1, 2, 3].

8.7 Manual list algorithms

Writing a few algorithms manually is useful even when Python offers built-ins. It forces you to define state, handle empty input, reason about distinctness, and state complexity.

EX-051 | Second-largest distinct value Challenge - 20 min

Find the second-largest distinct number in one pass without sorting. Return None when fewer than two distinct values exist.

EX-052 | Merge sorted lists Challenge - 25 min

Merge two ascending lists into one ascending list in O(n + m) time without calling sorted() on the combined result.

Input / output: [1, 4, 7] and [2, 2, 9] -> [1, 2, 2, 4, 7, 9].

8.8 Nested lists

A list can contain lists, forming a grid or table-like structure. Access matrix[row][column]. Avoid creating rows with [[0] * columns] * rows because every row then refers to the same inner list. Use a comprehension or loop to create independent rows.

```
rows = 3
columns = 4
matrix = [[0 for _ in range(columns)] for _ in range(rows)]
matrix[0][0] = 9
print(matrix)
```

Output

```
[[9, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

EX-053 | Matrix transpose Challenge - 20 min

Transpose a rectangular matrix using nested loops. For [[1, 2, 3], [4, 5, 6]], produce [[1, 4], [2, 5], [3, 6]].

Constraints: Do not use NumPy; Assume all rows have equal length

Chapter summary

KEY IDEAS

- Lists provide ordered storage and constant-time indexing.
- append() and pop() at the end are efficient; front or middle changes shift items.
- In-place methods such as sort() return None.
- Slicing and copy() create shallow lists.
- Repeated list membership can turn a loop into O(n^2) work.

COMMON MISTAKES

- Assigning result = values.sort() and expecting a list.
- Removing items while iterating over the same list.
- Using a repeated-inner-list expression for a matrix.
- Copying a nested list and assuming all inner objects are independent.

PRACTICE RULE: When a list algorithm contains repeated membership checks or front insertions, note the cost and reconsider the structure after learning sets and deques.

CHAPTER 09

Tuples and Sequence Unpacking

Represent fixed-position values and move several results through code safely.

LEARNING OUTCOMES

- Create tuples and distinguish them from parenthesized expressions.
- Use packing, unpacking, starred targets, and tuple returns.
- Recognize tuple immutability and the effect of nested mutable values.
- Use hashable tuples as composite identifiers where appropriate.

9.1 Tuple syntax and purpose

A tuple is an ordered, immutable sequence. It communicates that the sequence structure should not change. Tuples are useful for coordinates, fixed return values, composite keys, and records whose positions are well-known and limited. The comma creates a tuple; parentheses mainly improve readability.

```
point = (12.5, 8.0)
empty = ()
single = (42,)
not_a_tuple = (42)

print(type(single))
print(type(not_a_tuple))
```

Output

```
<class 'tuple'>
<class 'int'>
```

DESIGN RULE

Use a tuple when position has a stable meaning and mutation would be an error. Use a list when items are expected to be added, removed, reordered, or replaced.

EX-054 | Coordinate unpacking Recall - 6 min

Store latitude=17.385 and longitude=78.4867 in a tuple. Unpack the tuple into two names and print a formatted coordinate pair.

9.2 Packing and unpacking

Implicit packing and exact unpacking

```
record = "TXN-101", "South", 2500.0
transaction_id, region, amount = record

print(transaction_id)
print(region)
print(amount)
```

The number of targets must normally match the number of items. A starred target collects zero or more items into a list, making it useful for a known beginning and end with a variable middle.

```
values = (10, 20, 30, 40, 50)
first, *middle, last = values
print(first)
print(middle)
print(last)
```

Output

```
10
[20, 30, 40]
50
```

EX-055 | Header and tail unpacking Apply - 8 min

Unpack record=("TXN-9", "South", 1200.0, "CARD", "SUCCESS") into transaction_id, a list of middle fields, and status. Print each result.

9.3 Immutability is shallow

Tuple positions cannot be rebound, but a tuple can contain a mutable object. That inner object can still change. Immutability therefore describes the tuple structure, not a recursive guarantee about every object reachable from it.

```
record = ("C101", ["new", "priority"])
record[1].append("verified")
print(record)

# record[0] = "C102"  # TypeError
```

Output

```
('C101', ['new', 'priority', 'verified'])
```

HASHABILITY CONSEQUENCE

A tuple is hashable only when every contained value is hashable. A tuple containing a list cannot be a dictionary key or set member.

EX-056 | Return a fixed summary Apply - 10 min

Write bounds(values) that returns a tuple (minimum, maximum) and returns (None, None) for empty input. Unpack and print the result.

9.4 Tuple sequence operations

Operation	Meaning
values[i]	Read one item
values[a:b]	Create a new tuple slice
len(values)	Count items
x in values	Linear membership test
values.count(x)	Count equal occurrences
values.index(x)	Position of first equal occurrence
left + right	Create a concatenated tuple

EX-057 | Tuple statistics Recall - 8 min

For scores=(82, 91, 82, 75, 91, 82), print the number of scores, the count of 82, the first index of 91, and a reversed tuple.

9.5 Composite keys

A tuple of hashable values can represent a composite identifier such as (region, month), (customer_id, date), or (row, column). The meaning of each position must be documented. For many fields, a named record is easier to read than a plain positional tuple.

EX-058 | Composite period keys Apply - 10 min

Create tuples for ("South", 2026, 11) and ("South", 2026, 12). Show that they compare unequal, and sort a list containing both plus ("North", 2026, 12).

Chapter summary

KEY IDEAS

- The comma creates a tuple; a one-item tuple needs a trailing comma.
- Unpacking assigns positions to names and can collect a variable middle with *.
- Tuple immutability prevents position replacement but not mutation of nested objects.
- A tuple is hashable only when all contained values are hashable.

COMMON MISTAKES

- Using a long positional tuple whose field meaning is unclear.
- Forgetting the comma in a single-item tuple.
- Assuming a tuple recursively freezes nested lists or dictionaries.
- Unpacking without validating record length at an external boundary.

PRACTICE RULE: When returning a tuple, document the position and meaning of every result and keep the shape stable on all paths.

CHAPTER 10

Sets: Uniqueness and Fast Membership

Use hash-based collections for deduplication, comparison, and repeated existence checks.

LEARNING OUTCOMES

- Create sets and understand that elements must be hashable.
- Use union, intersection, difference, and symmetric difference.
- Replace repeated list membership with a set when order is not the primary requirement.
- Use a companion set to deduplicate while preserving first-seen order.

10.1 Set fundamentals

A set is an unordered collection of unique, hashable elements. Membership, insertion, and removal are O(1) on average because sets use a hash table. Order should not be treated as a stable contract even if a particular run appears to use a consistent order.

Duplicate elements collapse; {} creates a dictionary

```
skills = {"Python", "SQL", "Python"}
print(skills)
print("SQL" in skills)

empty_set = set()
empty_dict = {}
```

Core set operations

Method / operation	Meaning	Typical cost
add(x)	Add one element	avg O(1)
update(items)	Add several elements	avg O(k)
remove(x)	Remove x; fail if absent	avg O(1)
discard(x)	Remove x if present	avg O(1)
x in values	Membership test	avg O(1)
left | right	Union	O(n + m)
left & right	Intersection	O(min(n, m)) typical
left - right	Difference	O(n) typical
left ^ right	Symmetric difference	O(n + m)

EX-059 | Unique customer count Recall - 6 min

Given customer IDs ["C1", "C2", "C1", "C3", "C2", "C4"], print the unique IDs and unique count using a set.

10.2 Set algebra expresses comparison

Skill-gap analysis

```
required = {"Python", "SQL", "Excel"}
candidate = {"Python", "Power BI", "Excel"}

matched = required & candidate
missing = required - candidate
extra = candidate - required
all_skills = required | candidate

print(matched)
print(missing)
print(extra)
```

Set expressions often mirror the business question directly. Intersection answers "present in both"; difference answers "required but missing"; symmetric difference answers "present in exactly one".

EX-060 | Skill comparison Apply - 10 min

Compare required={"Python", "SQL", "Statistics", "Excel"} with candidate={"Python", "SQL", "Power BI"}. Print matched, missing, and additional skills in sorted order.

EX-061 | Changed identifiers Apply - 8 min

Yesterday IDs were {101, 102, 103, 104}; today they are {102, 103, 104, 105, 106}. Print added, removed, and unchanged IDs.

10.3 Subsets, supersets, and disjointness

Expression	Question
a <= b	Is a a subset of b?
a < b	Is a a proper subset of b?
a >= b	Is a a superset of b?
a.isdisjoint(b)	Do a and b have no common elements?

```
required_permissions = {"read", "export"}
user_permissions = {"read", "export", "comment"}
print(required_permissions <= user_permissions)
```

Output

```
True
```

EX-062 | Permission gate Apply - 10 min

A report requires {"read", "finance", "export"}. Determine whether a user with {"read", "finance", "export", "comment"} has every required permission. Also check whether the user has any forbidden permission from {"delete", "admin"}.

10.4 Order-preserving deduplication

Converting a list directly to a set removes duplicates but does not promise original order. To preserve first-seen order efficiently, use a result list plus a set of values already seen.

```
values = [4, 2, 4, 1, 2, 3, 1]
seen = set()
unique = []

for value in values:
    if value in seen:
        continue
    seen.add(value)
    unique.append(value)

print(unique)
```

Output

```
[4, 2, 1, 3]
```

EX-063 | Efficient ordered deduplication Apply - 12 min

Rewrite EX-048 using a companion set. State the average time and space complexity.

10.5 Hashability and frozenset

Set elements must be hashable: their hash and equality behavior must remain stable while stored. Strings, numbers, and tuples of hashable elements are common set members. Lists, dictionaries, and sets are mutable and therefore not hashable. frozenset is an immutable, hashable set variant.

A set can contain frozensets

```
required_groups = {
    frozenset({"Python", "SQL"}),
    frozenset({"Excel", "Power BI"}),
}
print(len(required_groups))
```

EX-064 | Duplicate transaction detector Challenge - 18 min

Transactions are tuples (customer_id, date, amount). Print every transaction that appears more than once, but print each duplicated transaction only once.

Chapter summary

KEY IDEAS

- Sets store unique hashable values and provide fast average membership.
- Set algebra expresses overlap and difference directly.
- A set does not provide a stable ordering contract.
- A companion set supports efficient ordered deduplication.
- frozenset is an immutable, hashable set.

COMMON MISTAKES

- Writing {} when an empty set was intended.
- Relying on set iteration order.
- Trying to add a list or dictionary to a set.
- Using a list for thousands of repeated membership checks.

PRACTICE RULE: When the core question is "have I seen this?" or "what is common or missing?", evaluate a set before writing nested loops.

CHAPTER 11

Dictionaries: Keyed Lookup and Aggregation

Model named fields, indexes, counters, and grouped results with hash-based mappings.

LEARNING OUTCOMES

- Create, access, update, remove, and iterate through dictionaries.
- Choose between indexing, get(), setdefault(), and explicit membership.
- Build counters and group-based aggregations.
- Explain key hashability and average lookup complexity.

11.1 Dictionary fundamentals

A dictionary maps unique hashable keys to values. Keys provide meaning and direct lookup, making dictionaries the natural structure for records, configurations, indexes, counters, and aggregations. Values can be any Python object and do not need to share a type.

Read, update, and insert

```
customer = {
    "customer_id": "C101",
    "name": "Anita Rao",
    "active": True,
    "balance": 1250.50,
}

print(customer["name"])
customer["balance"] += 500
customer["segment"] = "Gold
```

Core dictionary operations

Method / operation	Meaning	Typical cost
mapping[key]	Read value or raise KeyError	avg O(1)
mapping.get(key)	Read value or return None/default	avg O(1)
mapping[key] = value	Insert or replace	avg O(1)
key in mapping	Check key membership	avg O(1)
mapping.pop(key)	Remove and return a value	avg O(1)
mapping.keys()	Dynamic view of keys	O(1) to create
mapping.values()	Dynamic view of values	O(1) to create
mapping.items()	Dynamic view of key-value pairs	O(1) to create

EX-065 | Customer profile update Recall - 10 min

Create a customer dictionary with id, name, city, active, and balance. Update the city, add a segment, and safely remove a temporary field using pop(). Print all key-value pairs.

11.2 Indexing versus get()

Use mapping[key] when the key is required and absence indicates a defect. Use get() when absence is expected and a default has defined meaning. A default can hide bad data, so do not automatically replace every missing field with zero or an empty string.

```
record = {"amount": 250.0, "status": "approved"}

amount = record["amount"]             # required
comment = record.get("comment")       # optional -> None
currency = record.get("currency", "INR")

print(comment, currency)
```

Output

```
None INR
```

MISSING IS NOT ALWAYS ZERO

record.get("revenue", 0) silently converts an absent field into a real zero. Use that default only when the domain explicitly defines absence as zero.

EX-066 | Safe configuration lookup Apply - 8 min

Given config={"environment":"prod", "retries":3}, read required environment, optional timeout with default 30, and optional owner with default None. Print all three.

11.3 Counting frequencies

A frequency dictionary maps each distinct item to its count. The get(key, 0) pattern handles the first occurrence and later increments uniformly.

```
words = ["python", "sql", "python", "excel", "python"]
counts = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)
```

Output

```
{'python': 3, 'sql': 1, 'excel': 1}
```

EX-067 | Word-frequency table Apply - 12 min

Normalize the sentence "Python SQL python Excel SQL python" to lowercase, split it, and build a frequency dictionary. Print words in alphabetical order with their counts.

11.4 Aggregating by a key

A grouped aggregation maps a category to accumulated state. The state may be one total, a count and total, or a nested record of several metrics. Make the input grain and output grain explicit before coding.

```
sales = [
    ("South", 120.0),
    ("North", 80.0),
    ("South", 50.0),
]
totals = {}

for region, amount in sales:
    totals[region] = totals.get(region, 0.0) + amount

print(totals)
```

Output

```
{'South': 170.0, 'North': 80.0}
```

EX-068 | Sales by region Apply - 15 min

Aggregate transaction tuples into total revenue per region. Then print regions from highest total to lowest.

EX-069 | Count and average by category Challenge - 20 min

For records (category, value), build a dictionary whose value for each category contains count, total, and average. Avoid recomputing the total from stored raw values.

11.5 setdefault() and explicit initialization

setdefault(key, default) returns the existing value or inserts and returns the default. It is convenient for grouping, but explicit initialization is sometimes clearer when the state is complex. The default expression is still evaluated before the call, so avoid expensive defaults in hot loops.

```
groups = {}
for region, amount in [("South", 10), ("North", 20), ("South", 30)]:
    groups.setdefault(region, []).append(amount)

print(groups)
```

Output

```
{'South': [10, 30], 'North': [20]}
```

EX-070 | Group names by initial Apply - 12 min

Group ["Anita", "Arun", "Bhavna", "Bharat", "Chitra"] by first letter in a dictionary of lists. Sort each group alphabetically.

11.6 Maximum, sorting, and inversion

Select the key whose value is largest

```
totals = {"South": 170, "North": 120, "West": 200}
top_region = max(totals, key=totals.get)
print(top_region, totals[top_region])
```

Output

```
West 200
```

Inverting a mapping is safe only when values are unique and hashable. When several keys share a value, the inverse needs lists or sets to avoid overwriting earlier keys.

EX-071 | Invert with duplicate values Apply - 12 min

Invert {"A":"South", "B":"North", "C":"South"} into a dictionary mapping each region to a list of codes.

11.7 Merging dictionaries

The | operator creates a merged dictionary in modern Python. Later mappings win on duplicate keys. The update() method mutates the left dictionary. For quantities or counters, simple overwriting may be wrong; merge according to business meaning.

```
defaults = {"timeout": 30, "retries": 2, "format": "csv"}
overrides = {"retries": 5, "format": "json"}
settings = defaults | overrides
print(settings)
```

Output

```
{'timeout': 30, 'retries': 5, 'format': 'json'}
```

EX-072 | Merge inventory quantities Apply - 12 min

Merge warehouse_a={"P1":10, "P2":5} and warehouse_b={"P2":7, "P3":4} by adding quantities for duplicate product IDs.

11.8 Hashable keys and stable meaning

Keys must be hashable. Strings, numbers, and tuples of hashable fields are common. Prefer stable identifiers over display names that may change or collide. If a key is normalized, apply the same normalization at every lookup boundary.

EX-073 | Group anagrams Challenge - 20 min

Group words that are anagrams by using a sorted-character tuple as the dictionary key. For ["eat", "tea", "tan", "ate", "nat", "bat"], produce groups of equivalent words.

Chapter summary

KEY IDEAS

- Dictionaries map hashable keys to values with average O(1) lookup.
- Use [] for required keys and get() for optional keys with a defined default policy.
- Frequency counts and grouped totals are core dictionary patterns.
- Merge behavior must reflect the domain: overwrite, add, combine, or reject.
- Stable identifiers make better keys than changeable labels.

COMMON MISTAKES

- Using get(key, 0) when missing and zero have different meanings.
- Iterating only keys when both keys and values are required.
- Overwriting duplicate groups during inversion.
- Using an unhashable list or dictionary as a key.

PRACTICE RULE: When building an aggregation dictionary, write the input grain, output key, stored state, and update rule before the loop.

CHAPTER 12

Nested Data Structures and Record Modeling

Represent realistic data without losing clarity, validation, or control over missing fields.

LEARNING OUTCOMES

- Traverse lists of dictionaries and dictionaries of lists.
- Access nested fields safely while preserving missing-state meaning.
- Group, filter, join, and flatten record-like data.
- Recognize shallow-copy risks in nested mutable structures.

12.1 Common shapes

Shape	Typical meaning	Example
list[dict]	Rows or records	[{"id":"C1"}, {"id":"C2"}]
dict[list]	Columns or grouped values	{"amount":[10, 20]}
dict[dict]	Index by identifier	{"C1":{"city":"HYD"}}
list[tuple]	Compact fixed-position rows	[("South", 100), ...]

Choose the shape that matches the dominant access pattern. A list of records is easy to iterate. A dictionary indexed by ID is better for repeated direct lookup. Conversion between shapes has a cost, so avoid changing representation without a reason.

A list of record dictionaries

```
orders = [
    {"order_id": "O1", "region": "South", "amount": 120.0},
    {"order_id": "O2", "region": "North", "amount": 80.0},
]

for order in orders:
    print(order["order_id"], order["amount"])
```

EX-074 | Revenue from records Apply - 15 min

Given a list of order dictionaries, calculate total revenue, order count, and average order value. Reject records missing amount rather than treating them as zero.

12.2 Safe nested access

Chaining get() calls requires care because an intermediate value may be None or another unexpected type. For required schemas, explicit validation is clearer. For optional nested sections, retrieve one level at a time and define the fallback deliberately.

Optional nested objects

```
record = {
    "customer": {
        "id": "C1",
        "address": {"city": "Hyderabad"},
    }
}

customer = record.get("customer") or {}
address = customer.get("address") or {}
city = address.get("city")
print(city)
```

DO NOT HIDE SCHEMA DEFECTS

A chain of empty-dictionary fallbacks can make malformed records look valid. Use fallbacks only for fields that are genuinely optional; fail or quarantine records that violate required structure.

EX-075 | Nested city extractor Apply - 15 min

Write get_city(record) that returns the nested customer.address.city value or None when any optional level is missing. Return None for non-dictionary input.

12.3 Filtering and projection

Filtering chooses records; projection chooses fields. Keep the original record untouched when creating a derived result unless mutation is explicitly required.

Filter and project into new dictionaries

```
orders = [
    {"id": "O1", "status": "PAID", "amount": 100},
    {"id": "O2", "status": "PENDING", "amount": 80},
]
paid_summary = []

for order in orders:
    if order["status"] == "PAID":
        paid_summary.append({
            "id": order["id"],
            "amount": order["amount"],
        })

print(paid_summary)
```

EX-076 | Valid transaction projection Apply - 15 min

From transaction dictionaries, select only status="SUCCESS" and positive amounts. Produce new dictionaries containing only id and amount, with amount rounded to two decimals.

12.4 Grouping records

A dictionary of lists groups full records or selected values by a category. A dictionary of metric dictionaries is better when only aggregated state is required. Storing all raw records uses more memory but supports later detail inspection.

EX-077 | Group orders by region Apply - 15 min

Group order dictionaries by region into a dictionary of lists. Keep the complete original record in each group and print the count per region.

12.5 Joining records with an index

Joining two lists with nested loops costs O(n * m). Build a dictionary index for one side, then perform average O(1) lookups while scanning the other side. This is the same broad idea used by database hash joins.

Index one side before joining

```
customers = [
    {"id": "C1", "name": "Anita"},
    {"id": "C2", "name": "Bharat"},
]
orders = [
    {"id": "O1", "customer_id": "C2", "amount": 100},
]
customer_by_id = {customer["id"]: customer for customer in customers}

for order in orders:
    customer = customer_by_id.get(order["customer_id"])
    print(order["id"], customer["name"] if customer else None)
```

EX-078 | Customer-order enrichment Challenge - 20 min

Enrich orders with customer name using a dictionary index. Keep orders whose customer is missing, but set customer_name to None and count the unmatched rows.

12.6 Flattening and schema validation

Flattening moves selected nested fields into one record for export or analysis. Do not flatten blindly; one-to-many arrays can multiply rows and change the data grain. Before transformation, validate required keys, types, and relationships.

```
required_keys = {"id", "region", "amount"}
record = {"id": "O1", "region": "South", "amount": 100}
missing = required_keys - record.keys()
print(missing)
```

Output

```
set()
```

EX-079 | Schema validator Challenge - 25 min

Write validate_order(record) returning a list of error messages. Require a dictionary with non-empty string id and region, numeric non-negative amount, and status in {"NEW", "PAID", "CANCELLED"}.

12.7 Shallow versus deep copy

copy(), slicing, and dictionary.copy() are shallow. Nested lists and dictionaries remain shared. The copy.deepcopy() function recursively copies supported objects, but deep copying large structures is costly and can hide a poor ownership design. Prefer constructing the exact derived record you need.

Independent nested structures

```
from copy import deepcopy

original = {"id": "C1", "tags": ["new"]}
scenario = deepcopy(original)
scenario["tags"].append("priority")

print(original)
print(scenario)
```

Chapter summary

KEY IDEAS

- Choose a nested shape based on the dominant access pattern.
- Validate required structure instead of masking defects with empty defaults.
- Filter and project into new records when the source should remain unchanged.
- Use a dictionary index to replace nested-loop joins.
- Shallow copies still share nested objects.

COMMON MISTAKES

- Chaining get() calls without verifying intermediate types.
- Using nested loops for repeated ID lookup when an index is appropriate.
- Flattening one-to-many data without defining the resulting grain.
- Assuming dict.copy() duplicates nested lists.

PRACTICE RULE: Before processing records, write a tiny schema: required keys, optional keys, types, allowed values, and missing-value policy.

CHAPTER 13

Comprehensions and Generator Expressions

Write concise transformations without hiding complicated logic.

LEARNING OUTCOMES

- Build list, set, and dictionary comprehensions.
- Distinguish filtering conditions from conditional expressions.
- Use nested comprehensions only when the reading order remains clear.
- Use generator expressions for one-pass, memory-efficient consumption.

13.1 The comprehension pattern

A comprehension combines result expression, iteration, and an optional filter. It is ideal for one clear transformation. When the logic requires several assignments, error handling, logging, or multiple branches, a normal loop is usually easier to read and debug.

```
values = [1, 2, 3, 4, 5, 6]
even_squares = [value ** 2 for value in values if value % 2 == 0]
print(even_squares)
```

Output

```
[4, 16, 36]
```

Part	Example	Purpose
Expression	value ** 2	Value placed in the result
for clause	for value in values	Source iteration
if filter	if value % 2 == 0	Keep only matching items

EX-080 | Even squares Recall - 6 min

Use a list comprehension to produce squares of even integers from 1 through 20 inclusive.

13.2 Filtering versus conditional output

```
values = [-2, 0, 3]

# Filter: result contains fewer items.
positive = [value for value in values if value > 0]

# Conditional expression: result has one output per input.
labels = ["positive" if value > 0 else "not positive" for value in values]

print(positive)
print(labels)
```

Output

```
[3]
['not positive', 'not positive', 'positive']
```

READING RULE

The conditional expression appears before for. A filtering if appears after the for clause. Mixing the two without formatting can make code difficult to scan.

EX-081 | Normalized unique skills Apply - 10 min

Use a set comprehension to strip whitespace, casefold values, and exclude empty strings from [" Python ", "SQL", "python", "", " Excel "].

13.3 Dictionary comprehensions

A dictionary comprehension has key and value expressions. Duplicate generated keys overwrite earlier values, so it is not a replacement for aggregation unless overwriting is explicitly intended.

```
products = [("P1", 5, 100), ("P2", 3, 80)]
revenue_by_product = {
    product_id: quantity * price
    for product_id, quantity, price in products
}
print(revenue_by_product)
```

Output

```
{'P1': 500, 'P2': 240}
```

EX-082 | Product revenue map Apply - 12 min

From rows=(product_id, quantity, unit_price), build a dictionary mapping each product to row revenue. Assume product IDs are unique and exclude rows with negative quantity or price.

13.4 Nested comprehensions

Nested comprehensions read in the same order as equivalent nested loops. They are useful for a simple flattening or grid construction, but become opaque when conditions or transformations are complex.

```
matrix = [[1, 2, 3], [4, 5, 6]]
flat = [value for row in matrix for value in row]
print(flat)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

EX-083 | Flatten valid readings Apply - 10 min

Flatten [[10, None, 12], [8, -1], [15, 16]] into one list containing only values that are not None and non-negative.

13.5 Generator expressions

A generator expression uses parentheses and produces values lazily as the consumer requests them. It avoids building a full intermediate list, which is valuable for large streams or when a function such as sum(), min(), max(), any(), or all() consumes the values once.

```
values = [10, 20, 30, 40]
total_of_squares = sum(value ** 2 for value in values)
print(total_of_squares)
```

Output

```
3000
```

ONE-PASS BEHAVIOR

A generator is normally exhausted after iteration. Use a list when values must be revisited, indexed, or shared with several consumers.

EX-084 | Record transformation comprehension Challenge - 18 min

From transaction dictionaries, create a list of new dictionaries containing id and net_amount for successful transactions. net_amount is amount minus fee. Exclude non-positive net amounts.

Chapter summary

KEY IDEAS

- Use comprehensions for one clear transformation or filter.
- A filter removes inputs; a conditional expression produces one result per input.
- Dictionary comprehensions overwrite duplicate keys.
- Nested comprehensions follow nested-loop order but should remain simple.
- Generator expressions are lazy and generally one-pass.

COMMON MISTAKES

- Packing complex validation, logging, and exception handling into a comprehension.
- Using a dictionary comprehension when duplicate keys require aggregation.
- Forgetting that a generator is exhausted after consumption.
- Writing a nested comprehension that requires mental backtracking to understand.

PRACTICE RULE: If a comprehension cannot be explained in one sentence, expand it into a normal loop with named steps.

CHAPTER 14

The collections Module

Use specialized standard-library containers for counting, grouping, queues, and readable records.

LEARNING OUTCOMES

- Use Counter for frequencies and multiset operations.
- Use defaultdict for grouped or accumulated values.
- Use deque for queues, stacks, and bounded windows.
- Use namedtuple when a small immutable positional record needs field names.

14.1 Counter

collections.Counter is a dictionary subclass designed for counting hashable items. Missing keys return zero during indexing, and most_common() returns ranked item-count pairs. Counter arithmetic can combine or compare frequency collections.

```
from collections import Counter

statuses = ["OK", "FAILED", "OK", "PENDING", "OK"]
counts = Counter(statuses)
print(counts)
print(counts.most_common(2))
```

Output

```
Counter({'OK': 3, 'FAILED': 1, 'PENDING': 1})
[('OK', 3), ('FAILED', 1)]
```

EX-085 | Status frequency with Counter Apply - 12 min

Normalize status strings by stripping and uppercasing, exclude empty values, and use Counter to print frequencies and the most common status.

EX-086 | Inventory requirement gap Challenge - 12 min

Use Counter to compare required items ["P1", "P1", "P2", "P3"] with available items ["P1", "P2"]. Print the missing quantities as a Counter.

14.2 defaultdict

defaultdict(factory) calls the factory to create a missing value on first indexed access. list groups items, int creates zero for counting, and set creates an empty set. Be aware that merely reading a missing key with indexing inserts it into the mapping.

```
from collections import defaultdict

groups = defaultdict(list)
for region, amount in [("South", 10), ("North", 20), ("South", 30)]:
    groups[region].append(amount)

print(dict(groups))
```

Output

```
{'South': [10, 30], 'North': [20]}
```

EX-087 | Department grouping Apply - 12 min

Use defaultdict(list) to group employee names by department from (name, department) tuples. Sort each list and print a normal dictionary.

14.3 deque

collections.deque supports efficient append and pop operations at both ends. It is the standard choice for FIFO queues and is also useful for stacks and fixed-length rolling windows. A list is efficient as a stack at the right end but inefficient as a queue when pop(0) repeatedly shifts elements.

Operation	Meaning	Average cost
append(x)	Add to right	O(1)
appendleft(x)	Add to left	O(1)
pop()	Remove from right	O(1)
popleft()	Remove from left	O(1)
deque(maxlen=k)	Automatically retain final k items	O(1) per end update

EX-088 | Bounded moving average Apply - 15 min

Use deque(maxlen=3) to process [10, 20, 30, 40, 50]. After each new value, print the current window and its average.

14.4 namedtuple

namedtuple creates an immutable tuple subclass with named fields. It preserves tuple behavior while replacing unclear numeric indexes with attributes. For richer validation, defaults, methods, or evolving schemas, a dataclass is often more appropriate, but namedtuple remains useful for compact fixed records.

```
from collections import namedtuple

Sale = namedtuple("Sale", ["product", "quantity", "unit_price"])
sale = Sale("P1", 5, 100.0)
print(sale.product)
print(sale.quantity * sale.unit_price)
```

Output

```
P1
500.0
```

EX-089 | Readable immutable sale record Apply - 12 min

Create a namedtuple Sale with product_id, region, quantity, and unit_price. Instantiate two sales and calculate total revenue using attribute access.

Chapter summary

KEY IDEAS

- Counter expresses frequencies and multiset arithmetic.
- defaultdict creates missing values through a factory.
- deque provides O(1) operations at both ends and supports bounded windows.
- namedtuple adds field names to compact immutable records.

COMMON MISTAKES

- Using Counter when records need rich state beyond a count.
- Accidentally inserting a defaultdict key by reading it.
- Using list.pop(0) repeatedly for a large FIFO queue.
- Using a long namedtuple when a dictionary or dataclass would be clearer.

PRACTICE RULE: Use the simplest built-in collection first; move to collections only when its semantics make the code clearer or more efficient.

PART 3

Problem Solving

Complexity-aware structure choice and integrated analytics practice.

Chapters 15-16

CHAPTER 15

Complexity, Stacks, Queues, and Structure Choice

Choose containers from access patterns, not habit.

LEARNING OUTCOMES

- Use Big O notation to compare growth in time and memory.
- Implement stack and queue behavior with appropriate containers.
- Apply hash-map and set patterns to reduce repeated scanning.
- Use a heap when only the smallest or largest few items are needed.

15.1 Big O is a growth model

Big O describes how resource use grows as input size n increases. It ignores constants and lower-order terms to compare scalability. It does not predict exact runtime; data size, implementation, hardware, caching, and constant factors still matter.

Complexity	Typical interpretation	Example
O(1)	Constant work	List index, average dict lookup
O(log n)	Problem shrinks by a factor	Binary search in sorted data
O(n)	One pass	Sum a list, set construction
O(n log n)	Efficient comparison sorting	sorted(values)
O(n^2)	All pairs or nested full scans	List-based duplicate search
O(2^n)	Explore subsets	Naive combinatorial recursion

15.2 Operation comparison

Need	Prefer	Reason
Index by position	list / tuple	O(1) indexing
Fast membership	set	Average O(1) membership
Map identifier to record	dict	Average O(1) lookup
LIFO stack	list or deque	Efficient append/pop at right
FIFO queue	deque	Efficient append/popleft
Smallest/largest repeatedly	heapq	O(log n) push/pop
Preserve duplicates and order	list	Sequence semantics
Unique mathematical comparison	set	Set algebra

SELECTION QUESTION

State the dominant operations: lookup by key, membership, append, front removal, sorting, duplicate preservation, or range traversal. The correct structure follows from the operations and semantics.

15.3 Stack pattern

A stack is last-in, first-out. Use append() to push and pop() to remove the most recent item. Stacks support parsing, undo history, depth-first search, expression evaluation, and matching nested delimiters.

List as a stack

```
stack = []
stack.append("first")
stack.append("second")
print(stack.pop())  # second
```

EX-090 | Balanced brackets Challenge - 25 min

Write is_balanced(text) for (), [], and {} brackets. Ignore non-bracket characters. Return False for a closing bracket that does not match the most recent opening bracket.

15.4 Queue pattern

A queue is first-in, first-out. deque.append() enqueues at the right and deque.popleft() dequeues from the left. Queues model task scheduling, breadth-first search, event processing, and records awaiting validation.

EX-091 | Support-ticket queue Apply - 12 min

Use deque to process tickets ["T1", "T2", "T3"] in arrival order. While processing T2, append an urgent follow-up T4 to the right. Print the processing order.

15.5 Hash-map pattern: complement lookup

When a problem asks whether a matching counterpart has been seen, a dictionary or set can replace an inner scan. The classic two-sum pattern stores earlier values and checks whether the current value completes the target. This reduces average time from O(n^2) to O(n).

EX-092 | Two-sum indexes Challenge - 20 min

Return the indexes of two numbers whose sum equals a target. Process the list once and use a dictionary from value to earlier index. Return None when no pair exists.

Input / output: [2, 7, 11, 15], target=9 -> (0, 1).

15.6 Replace pairwise scanning with sets

Comparing every item in one list with every item in another is O(n * m). If the question is membership, convert one side to a set and scan the other. Construction is O(m), and the scan is O(n) on average.

EX-093 | Efficient overlap Apply - 15 min

Return unique values from first=[1,2,2,3,4,5] that also occur in second=[2,4,4,6]. Preserve first-seen order from first and use a set for membership.

15.7 Heaps for top-k problems

The heapq module implements a min-heap in a list. The smallest item is at index zero, and push or pop costs O(log n). For k much smaller than n, maintaining a heap of size k can be more memory-efficient than sorting all n items. heapq.nlargest() and nsmallest() provide convenient top-k operations.

```
import heapq

values = [20, 5, 80, 30, 60, 10]
print(heapq.nlargest(3, values))
```

Output

```
[80, 60, 30]
```

EX-094 | Top-k products Apply - 12 min

Given product-revenue pairs, use heapq.nlargest() to return the top three by revenue. Preserve product and revenue together.

Chapter summary

KEY IDEAS

- Complexity compares growth, not exact runtime.
- Choose structures from semantics and dominant operations.
- Use a list or deque for stacks and a deque for FIFO queues.
- Dictionary and set lookups often remove an inner scan.
- A heap is useful when k is small relative to the full input.

COMMON MISTAKES

- Claiming an entire algorithm is O(1) because one operation is O(1).
- Using list.pop(0) for a large queue.
- Sorting all values when only a tiny top-k is needed without considering trade-offs.
- Optimizing before measuring or before defining the correct result.

PRACTICE RULE: State time and extra-space complexity after correctness, and name the operation that dominates each bound.

CHAPTER 16

Integrated Python and Analytics Practice

Combine parsing, validation, functions, loops, and data structures into end-to-end solutions.

LEARNING OUTCOMES

- Decompose a multi-step problem into parsing, validation, transformation, aggregation, and reporting.
- Choose data structures based on the required access and output grain.
- Produce audit-friendly metrics and data-quality counts.
- Explain complexity, edge cases, and assumptions for complete solutions.

16.1 A repeatable solution workflow

6. Restate the input shape and output shape.
7. Write examples, boundaries, invalid cases, and duplicate policy.
8. Choose structures for lookup, uniqueness, order, and grouping.
9. Separate parsing and validation from aggregation.
10. Implement the simplest correct version.
11. Test normal, empty, duplicate, missing, and malformed inputs.
12. State time and extra-space complexity.

ANALYTICS LENS

A correct total without data-quality counts can still be misleading. Report how many records were accepted, rejected, unmatched, duplicated, or missing whenever those states affect trust.

EX-095 | Parse and aggregate transaction lines Challenge - 30 min

Process pipe-delimited lines id|region|amount|status. Accept only four-field rows with numeric non-negative amount and status SUCCESS. Return total revenue by region plus accepted and rejected counts.

EX-096 | Customer-order summary Challenge - 35 min

Given customer and order records, create one summary per customer containing order_count, total_revenue, and average_order_value. Include customers with no orders and count unmatched orders whose customer ID is unknown.

EX-097 | Duplicate and conflict detector Challenge - 35 min

Transactions have id, customer_id, and amount. Report exact duplicate records separately from conflicting duplicate IDs whose other fields differ. Preserve first-seen records in an index.

EX-098 | Efficient rolling average Challenge - 30 min

Write rolling_average(values, window_size) using deque and a running sum. Return one average for each complete window and reject non-positive window sizes.

EX-099 | Inventory reconciliation Challenge - 35 min

Compare ERP stock and physical stock dictionaries by product. Produce a record for every product appearing in either source with erp, physical, variance=physical-erp, and status MATCH, SHORT, or EXCESS. Treat a missing side as None, not zero.

EX-100 | Product performance ranking capstone Challenge - 45 min

Given sales records with product, units, revenue, and returned_units, aggregate by product. Calculate return_rate, rank products by revenue descending, and flag HIGH_RETURN when return_rate exceeds 10 percent. Return a list of report dictionaries with rank. Validate that returned_units is not greater than units.

16.2 Final explanation checklist

BEFORE CALLING A SOLUTION COMPLETE

- Can you state the input and output grain?
- What happens for empty input, missing fields, duplicates, and invalid types?
- Which structure provides lookup, uniqueness, ordering, grouping, or queue behavior?
- What are the time and extra-space complexities?
- Which intermediate counts make the result auditable?
- Can each function be tested independently?

Chapter summary

KEY IDEAS

- Separate boundary parsing, validation, transformation, aggregation, and reporting.
- Initialize outputs from the entity population when zero-activity entities must remain visible.
- Distinguish exact duplicates from conflicting identifiers.
- Use rolling state instead of repeated full-window work.
- Report data-quality counts alongside business metrics.

COMMON MISTAKES

- Silently dropping invalid or unmatched records without counts.
- Treating missing source data as a numeric zero.
- Mixing parsing, aggregation, and display in one untestable block.
- Ranking row-level data before aggregating to the intended grain.

PRACTICE RULE: Re-solve EX-095 through EX-100 one week later from blank files. Explain every data-structure choice and test at least five edge cases per problem.

PART 4

Reference and Mastery

Use the solutions only after a genuine attempt.

Appendix A: Reference Solutions | Appendix B: Core Cheat Sheet | Appendix C: Glossary | Appendix D: Readiness Assessment

Appendix A: Reference solutions

A reference solution is one correct approach, not the only approach. Compare behavior, edge cases, readability, and complexity rather than copying surface syntax. Re-type any solution you study and then modify one requirement to prove that you understand it.

Solutions 001-010

EX-001 - First reproducible script

This exercise establishes the edit-save-run loop and demonstrates that variable values can be passed to print().

Reference solution

```
name = "Koushik"
target_role = "Data Analyst"
weekly_hours = 10

print("Learner:", name)
print("Target role:", target_role)
print("Weekly study hours:", weekly_hours)
```

EX-002 - Console receipt

The subtotal is derived from source values, so changing quantity or price updates the output correctly.

Reference solution

```
item = "Notebook"
quantity = 3
unit_price = 45.50
subtotal = quantity * unit_price

print("RECEIPT")
print("Item:", item)
print("Quantity:", quantity)
print(f"Unit price: {unit_price:.2f}")
print(f"Subtotal: {subtotal:.2f}")
```

EX-003 - Minutes to seconds

The conversion creates an integer that can participate in arithmetic.

Reference solution

```
minutes_text = input("Minutes: ")
minutes = int(minutes_text)
seconds = minutes * 60
print(f"{minutes} minutes = {seconds} seconds")
```

EX-004 - Repair a broken invoice

float() converts the numeric text. The formatted string controls display precision without changing the underlying value.

Reference solution

```
price_text = "125.00"
quantity = 2
price = float(price_text)
total = price * quantity
print(f"{total:.2f}")
```

EX-005 - Safe swap

Python evaluates the right side first and then unpacks the two values into the left-side names.

Reference solution

```
first = "AP"
second = "TS"
first, second = second, first
print(first, second)
```

EX-006 - Trace an alias

Assignment does not copy a list. Mutating through either reference changes the one shared object.

Reference solution

```
original = [5, 10]
alias = original
alias.append(15)

print(original)
print(alias)
print(id(original), id(alias))
print("Both names reference the same mutable list.")
```

EX-007 - Independent scenario list

The two flat lists are independent because list.copy() created a new outer list.

Reference solution

```
baseline = [100, 120, 90]
scenario = baseline.copy()
scenario[1] = 150

print("Baseline:", baseline)
print("Scenario:", scenario)
```

EX-008 - Equality and identity report

Equal dictionaries can have different identities. The alias is both equal to and identical with first.

Reference solution

```
first = {"region": "South", "sales": 500}
second = {"region": "South", "sales": 500}
alias = first

print("first == second:", first == second)
print("first is second:", first is second)
print("first == alias:", first == alias)
print("first is alias:", first is alias)
```

EX-009 - Object inspection helper

repr() exposes quotes and escape characters, while type and identity help distinguish values that may all be false in conditions.

Reference solution

```
def inspect_value(value):
    print("repr:", repr(value))
    print("type:", type(value).__name__)
    print("id:", id(value))
    print("is None:", value is None)
    print()

inspect_value(0)
inspect_value("")
inspect_value(None)
inspect_value([])
```

EX-010 - Temperature converter

Reference solution

```
celsius = 37
fahrenheit = celsius * 9 / 5 + 32
print(f"{celsius} C = {fahrenheit:.1f} F")
```

Solutions 011-020

EX-011 - Seconds to clock time

divmod(a, b) returns the quotient and remainder together and expresses the conversion clearly.

Reference solution

```
total_seconds = 9_125
hours, remainder = divmod(total_seconds, 3_600)
minutes, seconds = divmod(remainder, 60)
print(f"{hours:02d}:{minutes:02d}:{seconds:02d}")
```

EX-012 - Eligibility rule

Reference solution

```
amount = 7_500
account_active = True
is_blocked = False

eligible = 500 <= amount <= 20_000 and account_active and not is_blocked
print(eligible)
```

EX-013 - Divisibility classifier

Named Boolean expressions make the later decision easier to read and test.

Reference solution

```
number = 18
by_two = number % 2 == 0
by_three = number % 3 == 0

if by_two and by_three:
    print("divisible by both")
elif by_two:
    print("divisible by 2")
elif by_three:
    print("divisible by 3")
else:
    print("divisible by neither")
```

EX-014 - Invoice total

Each intermediate name makes the business formula auditable and reduces the chance of applying tax to the wrong base.

Reference solution

```
quantity = 4
unit_price = 250.0
discount_pct = 10.0
tax_pct = 18.0

subtotal = quantity * unit_price
discount = subtotal * discount_pct / 100
discounted = subtotal - discount
tax = discounted * tax_pct / 100
total = discounted + tax

print(f"Subtotal: {subtotal:.2f}")
print(f"Discount: {discount:.2f}")
print(f"Tax: {tax:.2f}")
print(f"Total: {total:.2f}")
```

EX-015 - Tolerance-based reconciliation

An absolute tolerance matches a business rule stated in currency units.

Reference solution

```
from math import isclose

left = 125_000.10
right = 125_000.11
difference = abs(left - right)
reconciled = isclose(left, right, rel_tol=0.0, abs_tol=0.02)

print(f"Difference: {difference:.2f}")
print("Reconciled:", reconciled)
```

EX-016 - KPI calculator

The code distinguishes an unavailable metric from a real numeric zero.

Reference solution

```
revenue = 500_000.0
cost = 350_000.0
order_count = 0

margin_pct = None if revenue == 0 else (revenue - cost) / revenue * 100
average_order_value = None if order_count == 0 else revenue / order_count

print("Margin %:", margin_pct)
print("Average order value:", average_order_value)
```

EX-017 - Initials extractor

Reference solution

```
first_name = "Ada"
last_name = "Lovelace"
initials = first_name[0] + last_name[0]
print(initials.upper())
```

EX-018 - Customer-name normalizer

split() without an argument removes surrounding whitespace and collapses repeated whitespace before join() rebuilds one-space separation.

Reference solution

```
raw_name = "   aNITA    rAO  "
matching_name = " ".join(raw_name.split()).casefold()
display_name = matching_name.title()

print("Raw:", repr(raw_name))
print("Matching:", matching_name)
print("Display:", display_name)
```

EX-019 - Email components

maxsplit=1 prevents an accidental extra split from producing too many values.

Reference solution

```
email = "analyst@example.com"
local_part, domain = email.split("@", maxsplit=1)

print("Local:", local_part)
print("Domain:", domain)
print("Commercial domain:", domain.endswith(".com"))
```

EX-020 - Transaction-line parser

Reference solution

```
line = "TXN-9|South|2500.75|approved"
transaction_id, region, amount_text, status = line.split("|")
amount = float(amount_text)
status = status.upper()

print(f"{transaction_id}: {region} | {amount:.2f} | {status}")
```

Solutions 021-030

EX-021 - File-name builder

Reference solution

```
region = " South "
report = "Sales"
year = 2026
month = 11

region_key = region.strip().casefold().replace(" ", "_")
report_key = report.strip().casefold().replace(" ", "_")
file_name = f"{region_key}_{report_key}_{year}_{month:02d}.csv"
print(file_name)
```

EX-022 - Simple customer-ID validator

The rule is decomposed into length, prefix, and suffix checks.

Reference solution

```
customer_id = "CU004219"
is_valid = (
    len(customer_id) == 8
    and customer_id.startswith("CU")
    and customer_id[2:].isdecimal()
)
print(is_valid)
```

EX-023 - Fixed-width report row

Reference solution

```
product = "Laptop"
units = 128
revenue = 8_745_250.5
margin = 0.1834

print(f"{product:<15} {units:>8d} {revenue:>16,.2f} {margin:>9.1%}")
```

EX-024 - Account-number masker

The number of masking characters is derived from the input length.

Reference solution

```
account = "123456789012"
visible = 4
masked = "*" * (len(account) - visible) + account[-visible:]
print(masked)
```

EX-025 - Number sign classifier

Reference solution

```
number = -8

if number > 0:
    print("positive")
elif number < 0:
    print("negative")
else:
    print("zero")
```

EX-026 - Grade band

Reference solution

```
score = 82

if not 0 <= score <= 100:
    grade = "invalid"
elif score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(grade)
```

Quick checks

```
for test_score in [-1, 0, 59, 60, 79, 80, 89, 90, 100, 101]:
    score = test_score
    # Run the same decision logic and inspect the result.
```

EX-027 - Shipping-fee rule

The rule is clearer when base shipping and the remote surcharge are calculated separately.

Reference solution

```
order_value = 800
is_premium = True
is_remote = True

free_base_shipping = order_value >= 1_000 or is_premium
base_fee = 0 if free_base_shipping else 80
remote_fee = 120 if is_remote else 0
shipping_fee = base_fee + remote_fee

print(shipping_fee)
```

EX-028 - Record-quality status

Reference solution

```
amount = None

if amount is None:
    status = "missing"
elif amount < 0:
    status = "invalid"
elif amount == 0:
    status = "zero"
else:
    status = "valid"

print(status)
```

EX-029 - Leap-year decision

Reference solution

```
year = 2024
is_leap = year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
print(is_leap)
```

Quick checks

```
for year in [1900, 2000, 2024, 2025]:
    is_leap = year % 400 == 0 or (
        year % 4 == 0 and year % 100 != 0
    )
    print(year, is_leap)
```

EX-030 - Risk classification

The high-risk exceptions are checked before the broader amount rule. Reversing the order could incorrectly label a watchlisted high-value transaction as MEDIUM.

Reference solution

```
verified = True
amount = 75_000
on_watchlist = False

if not verified:
    risk = "HIGH"
elif on_watchlist:
    risk = "HIGH"
elif amount > 50_000:
    risk = "MEDIUM"
else:
    risk = "LOW"

print(risk)
```

Solutions 031-040

EX-031 - Sum without sum()

Reference solution

```
values = [12, 5, 8, 20]
total = 0

for value in values:
    total += value

print(total)
```

EX-032 - Factorial

The accumulator begins at the multiplicative identity 1. An empty range for n=0 leaves the correct value unchanged.

Reference solution

```
n = 5
factorial = 1

for value in range(2, n + 1):
    factorial *= value

print(factorial)
```

EX-033 - Revenue rows

Reference solution

```
products = ["A", "B", "C"]
quantities = [3, 5, 2]
prices = [100, 80, 150]

for row_no, (product, quantity, price) in enumerate(
    zip(products, quantities, prices), start=1
):
    revenue = quantity * price
    print(row_no, product, revenue)
```

EX-034 - Valid-value summary

The explicit None check comes before numeric comparison, and continue skips invalid records without nesting the main update.

Reference solution

```
values = [120, -5, None, 0, 75, 210]
valid_count = 0
valid_total = 0

for value in values:
    if value is None or value < 0:
        continue
    valid_count += 1
    valid_total += value

print(valid_count, valid_total)
```

EX-035 - First threshold breach

Reference solution

```
values = [30, 75, 90, 140, 200]

for index, value in enumerate(values):
    if value > 100:
        print(index, value)
        break
else:
    print("none")
```

EX-036 - Digit count

Reference solution

```
number = 50_720

if number == 0:
    digit_count = 1
else:
    digit_count = 0
    remaining = number
    while remaining > 0:
        digit_count += 1
        remaining //= 10

print(digit_count)
```

EX-037 - Multiplication grid

Reference solution

```
size = 5

for row in range(1, size + 1):
    for column in range(1, size + 1):
        print(f"{row * column:4d}", end="")
    print()
```

EX-038 - Name formatter

Reference solution

```
def format_name(first, last):
    clean_first = " ".join(first.split()).title()
    clean_last = " ".join(last.split()).title()
    return f"{clean_last}, {clean_first}"

print(format_name("  anita ", " rAO "))
```

EX-039 - BMI calculator

Reference solution

```
def calculate_bmi(weight_kg, height_m):
    if weight_kg <= 0 or height_m <= 0:
        raise ValueError("weight and height must be positive")
    return weight_kg / (height_m ** 2)

bmi = calculate_bmi(70, 1.75)
print(f"{bmi:.2f}")
```

EX-040 - Safe percentage

Reference solution

```
def safe_percentage(part, whole):
    if part < 0 or whole < 0:
        raise ValueError("inputs cannot be negative")
    if whole == 0:
        return None
    return part / whole * 100

print(safe_percentage(45, 200))
print(safe_percentage(0, 0))
```

Solutions 041-050

EX-041 - Manual numeric summary

The first actual value initializes minimum and maximum, avoiding an arbitrary sentinel.

Reference solution

```
def summarize(values):
    if not values:
        return 0, 0, None, None, None

    total = 0
    minimum = values[0]
    maximum = values[0]

    for value in values:
        total += value
        if value < minimum:
            minimum = value
        if value > maximum:
            maximum = value

    count = len(values)
    mean = total / count
    return count, total, minimum, maximum, mean

print(summarize([4, 9, 2, 7]))
```

EX-042 - Invoice function

Reference solution

```
def calculate_invoice(
    quantity,
    unit_price,
    *,
    discount_pct=0,
    tax_pct=0,
):
    values = [quantity, unit_price, discount_pct, tax_pct]
    if any(value < 0 for value in values):
        raise ValueError("inputs cannot be negative")

    subtotal = quantity * unit_price
    discount = subtotal * discount_pct / 100
    taxable = subtotal - discount
    tax = taxable * tax_pct / 100
    total = taxable + tax
    return subtotal, discount, tax, total

result = calculate_invoice(
    4, 250, discount_pct=10, tax_pct=18
)
print(result)
```

EX-043 - Reusable ID validator

Reference solution

```
def is_valid_customer_id(customer_id):
    if not isinstance(customer_id, str):
        return False
    return (
        len(customer_id) == 8
        and customer_id.startswith("CU")
        and customer_id[2:].isdecimal()
    )

print(is_valid_customer_id("CU004219"))
print(is_valid_customer_id(None))
```

EX-044 - Configurable discount

Reference solution

```
def apply_discount(
    amount,
    *,
    percent=0,
    minimum_total=0,
):
    if amount < 0 or minimum_total < 0:
        raise ValueError("amounts cannot be negative")
    if not 0 <= percent <= 100:
        raise ValueError("percent must be between 0 and 100")

    if amount < minimum_total:
        return amount
    return amount * (1 - percent / 100)

print(apply_discount(1500, percent=10, minimum_total=1000))
```

EX-045 - List editing drill

Reference solution

```
regions = ["North", "East", "West"]
regions.append("South")
regions.insert(2, "Central")
regions[1] = "Northeast"
regions.remove("West")
print(regions)
```

EX-046 - Scenario isolation

Reference solution

```
baseline = [120, 150, 90, 200]
scenario = baseline.copy()

for index, value in enumerate(scenario):
    scenario[index] = value * 1.10

print("Baseline:", baseline)
print("Scenario:", scenario)
```

EX-047 - Clean amount list

The conversion boundary handles malformed text without mixing it with valid numeric values.

Reference solution

```
raw = ["120.5", "", " 80 ", "invalid", "0"]
clean = []

for item in raw:
    text = item.strip()
    if not text:
        continue
    try:
        clean.append(float(text))
    except ValueError:
        continue

print(clean)
```

EX-048 - Order-preserving deduplication

The result is correct but repeated linear membership tests can create quadratic work.

Reference solution

```
values = [4, 2, 4, 1, 2, 3, 1]
unique = []

for value in values:
    if value not in unique:
        unique.append(value)

print(unique)
# Worst-case time: O(n^2) because membership scans a growing list.
```

EX-049 - Top three distinct values

Later, a set will make deduplication more direct. This version reinforces list behavior.

Reference solution

```
values = [10, 40, 20, 40, 30, 10, 50]
unique = []

for value in values:
    if value not in unique:
        unique.append(value)

result = sorted(unique, reverse=True)[:3]
print(result)
```

EX-050 - Rotate right

Modulo reduces any rotation to the equivalent number of positions within the list length.

Reference solution

```
values = [1, 2, 3, 4, 5]
k = 2

if values:
    k %= len(values)
    rotated = values[-k:] + values[:-k] if k else values.copy()
else:
    rotated = []

print(rotated)
```

Solutions 051-060

EX-051 - Second-largest distinct value

The loop tracks the two best distinct values. Each element is processed once, so time is O(n) and extra space is O(1).

Reference solution

```
values = [10, 5, 20, 20, 8]
largest = None
second = None

for value in values:
    if largest is None or value > largest:
        if value != largest:
            second = largest
            largest = value
    elif value != largest and (second is None or value > second):
        second = value

print(second)
```

EX-052 - Merge sorted lists

Each pointer moves only forward. The remaining suffix can be appended after either list is exhausted.

Reference solution

```
left = [1, 4, 7]
right = [2, 2, 9]
i = 0
j = 0
merged = []

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        merged.append(left[i])
        i += 1
    else:
        merged.append(right[j])
        j += 1

merged.extend(left[i:])
merged.extend(right[j:])
print(merged)
```

EX-053 - Matrix transpose

The outer loop chooses each original column, and the inner loop gathers that column across rows.

Reference solution

```
matrix = [[1, 2, 3], [4, 5, 6]]

if not matrix:
    transposed = []
else:
    row_count = len(matrix)
    column_count = len(matrix[0])
    transposed = []

    for column in range(column_count):
        new_row = []
        for row in range(row_count):
            new_row.append(matrix[row][column])
        transposed.append(new_row)

print(transposed)
```

EX-054 - Coordinate unpacking

Reference solution

```
location = (17.385, 78.4867)
latitude, longitude = location
print(f"({latitude:.4f}, {longitude:.4f})")
```

EX-055 - Header and tail unpacking

Reference solution

```
record = (
    "TXN-9",
    "South",
    1200.0,
    "CARD",
    "SUCCESS",
)
transaction_id, *details, status = record
print(transaction_id)
print(details)
print(status)
```

EX-056 - Return a fixed summary

Reference solution

```
def bounds(values):
    if not values:
        return None, None
    return min(values), max(values)

minimum, maximum = bounds([7, 2, 9, 4])
print(minimum, maximum)
```

EX-057 - Tuple statistics

Reference solution

```
scores = (82, 91, 82, 75, 91, 82)
print(len(scores))
print(scores.count(82))
print(scores.index(91))
print(scores[::-1])
```

EX-058 - Composite period keys

Tuple comparison is lexicographic: Python compares the first position, then the next when needed.

Reference solution

```
november = ("South", 2026, 11)
december = ("South", 2026, 12)
periods = [december, ("North", 2026, 12), november]

print(november == december)
print(sorted(periods))
```

EX-059 - Unique customer count

Reference solution

```
customer_ids = ["C1", "C2", "C1", "C3", "C2", "C4"]
unique_ids = set(customer_ids)
print(unique_ids)
print(len(unique_ids))
```

EX-060 - Skill comparison

Sorting is for deterministic display; it does not change the underlying set semantics.

Reference solution

```
required = {"Python", "SQL", "Statistics", "Excel"}
candidate = {"Python", "SQL", "Power BI"}

print("Matched:", sorted(required & candidate))
print("Missing:", sorted(required - candidate))
print("Additional:", sorted(candidate - required))
```

Solutions 061-070

EX-061 - Changed identifiers

Reference solution

```
yesterday = {101, 102, 103, 104}
today = {102, 103, 104, 105, 106}

print("Added:", today - yesterday)
print("Removed:", yesterday - today)
print("Unchanged:", yesterday & today)
```

EX-062 - Permission gate

Reference solution

```
required = {"read", "finance", "export"}
user = {"read", "finance", "export", "comment"}
forbidden = {"delete", "admin"}

has_required = required <= user
has_forbidden = not user.isdisjoint(forbidden)
print(has_required)
print(has_forbidden)
```

EX-063 - Efficient ordered deduplication

Each average-case membership check and insertion is O(1), replacing the growing list scan from the earlier solution.

Reference solution

```
values = [4, 2, 4, 1, 2, 3, 1]
seen = set()
unique = []

for value in values:
    if value not in seen:
        seen.add(value)
        unique.append(value)

print(unique)
# Average time: O(n); extra space: O(n).
```

EX-064 - Duplicate transaction detector

The transaction tuple is hashable because each field is hashable. The duplicates set suppresses repeated reporting.

Reference solution

```
transactions = [
    ("C1", "2026-11-01", 100.0),
    ("C2", "2026-11-01", 200.0),
    ("C1", "2026-11-01", 100.0),
    ("C1", "2026-11-02", 100.0),
    ("C2", "2026-11-01", 200.0),
]
seen = set()
duplicates = set()

for transaction in transactions:
    if transaction in seen:
        duplicates.add(transaction)
    else:
        seen.add(transaction)

for transaction in sorted(duplicates):
    print(transaction)
```

EX-065 - Customer profile update

Reference solution

```
customer = {
    "id": "C101",
    "name": "Anita Rao",
    "city": "Hyderabad",
    "active": True,
    "balance": 1250.0,
    "temporary": "remove me",
}
customer["city"] = "Bengaluru"
customer["segment"] = "Gold"
customer.pop("temporary", None)

for key, value in customer.items():
    print(key, value)
```

EX-066 - Safe configuration lookup

Reference solution

```
config = {"environment": "prod", "retries": 3}
environment = config["environment"]
timeout = config.get("timeout", 30)
owner = config.get("owner")
print(environment, timeout, owner)
```

EX-067 - Word-frequency table

Reference solution

```
sentence = "Python SQL python Excel SQL python"
words = sentence.casefold().split()
counts = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

for word in sorted(counts):
    print(word, counts[word])
```

EX-068 - Sales by region

Sorting items with a value-based key produces a ranked presentation while retaining the dictionary for lookup.

Reference solution

```
transactions = [
    ("South", 120.0),
    ("North", 80.0),
    ("South", 50.0),
    ("West", 200.0),
    ("North", 40.0),
]
totals = {}

for region, amount in transactions:
    totals[region] = totals.get(region, 0.0) + amount

for region, total in sorted(
    totals.items(), key=lambda item: item[1], reverse=True
):
    print(region, total)
```

EX-069 - Count and average by category

The aggregation state stores only the sufficient statistics needed for the mean.

Reference solution

```
records = [
    ("A", 10),
    ("B", 20),
    ("A", 30),
    ("B", 40),
    ("A", 50),
]
summary = {}

for category, value in records:
    if category not in summary:
        summary[category] = {"count": 0, "total": 0}
    summary[category]["count"] += 1
    summary[category]["total"] += value

for metrics in summary.values():
    metrics["average"] = metrics["total"] / metrics["count"]

print(summary)
```

EX-070 - Group names by initial

Reference solution

```
names = ["Anita", "Arun", "Bhavna", "Bharat", "Chitra"]
groups = {}

for name in names:
    groups.setdefault(name[0], []).append(name)

for values in groups.values():
    values.sort()

print(groups)
```

Solutions 071-080

EX-071 - Invert with duplicate values

Reference solution

```
code_to_region = {
    "A": "South",
    "B": "North",
    "C": "South",
}
region_to_codes = {}

for code, region in code_to_region.items():
    region_to_codes.setdefault(region, []).append(code)

print(region_to_codes)
```

EX-072 - Merge inventory quantities

Dictionary union would overwrite P2. The loop implements the required additive merge.

Reference solution

```
warehouse_a = {"P1": 10, "P2": 5}
warehouse_b = {"P2": 7, "P3": 4}
combined = warehouse_a.copy()

for product_id, quantity in warehouse_b.items():
    combined[product_id] = combined.get(product_id, 0) + quantity

print(combined)
```

EX-073 - Group anagrams

Anagrams produce the same sorted tuple, which is hashable and therefore suitable as a dictionary key.

Reference solution

```
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
groups = {}

for word in words:
    key = tuple(sorted(word))
    groups.setdefault(key, []).append(word)

print(list(groups.values()))
```

EX-074 - Revenue from records

Reference solution

```
orders = [
    {"order_id": "O1", "amount": 120.0},
    {"order_id": "O2", "amount": 80.0},
    {"order_id": "O3", "amount": 200.0},
]

total = 0.0
for order in orders:
    if "amount" not in order:
        raise ValueError(f"missing amount: {order}")
    total += order["amount"]

count = len(orders)
average = None if count == 0 else total / count
print(count, total, average)
```

EX-075 - Nested city extractor

Explicit type checks distinguish absent optional sections from an invalid object shape.

Reference solution

```
def get_city(record):
    if not isinstance(record, dict):
        return None
    customer = record.get("customer")
    if not isinstance(customer, dict):
        return None
    address = customer.get("address")
    if not isinstance(address, dict):
        return None
    return address.get("city")

print(get_city({"customer": {"address": {"city": "Hyderabad"}}}))
```

EX-076 - Valid transaction projection

Reference solution

```
transactions = [
    {"id": "T1", "status": "SUCCESS", "amount": 100.125},
    {"id": "T2", "status": "FAILED", "amount": 80.0},
    {"id": "T3", "status": "SUCCESS", "amount": -5.0},
    {"id": "T4", "status": "SUCCESS", "amount": 40.5},
]
result = []

for transaction in transactions:
    if (
        transaction.get("status") == "SUCCESS"
        and transaction.get("amount", 0) > 0
    ):
        result.append({
            "id": transaction["id"],
            "amount": round(transaction["amount"], 2),
        })

print(result)
```

EX-077 - Group orders by region

Reference solution

```
orders = [
    {"id": "O1", "region": "South", "amount": 100},
    {"id": "O2", "region": "North", "amount": 80},
    {"id": "O3", "region": "South", "amount": 50},
]
groups = {}

for order in orders:
    region = order["region"]
    groups.setdefault(region, []).append(order)

for region, region_orders in groups.items():
    print(region, len(region_orders))
```

EX-078 - Customer-order enrichment

The dictionary index avoids a repeated full scan of customers for each order.

Reference solution

```
customers = [
    {"id": "C1", "name": "Anita"},
    {"id": "C2", "name": "Bharat"},
]
orders = [
    {"id": "O1", "customer_id": "C2", "amount": 100},
    {"id": "O2", "customer_id": "C9", "amount": 80},
]
customer_by_id = {customer["id"]: customer for customer in customers}
enriched = []
unmatched = 0

for order in orders:
    customer = customer_by_id.get(order["customer_id"])
    if customer is None:
        unmatched += 1
    enriched.append({
        **order,
        "customer_name": None if customer is None else customer["name"],
    })

print(enriched)
print("Unmatched:", unmatched)
```

EX-079 - Schema validator

Returning all detected errors is more useful for data-quality reporting than stopping at the first problem.

Reference solution

```
def validate_order(record):
    errors = []
    if not isinstance(record, dict):
        return ["record must be a dictionary"]

    order_id = record.get("id")
    region = record.get("region")
    amount = record.get("amount")
    status = record.get("status")

    if not isinstance(order_id, str) or not order_id.strip():
        errors.append("id must be a non-empty string")
    if not isinstance(region, str) or not region.strip():
        errors.append("region must be a non-empty string")
    if not isinstance(amount, (int, float)) or isinstance(amount, bool):
        errors.append("amount must be numeric")
    elif amount < 0:
        errors.append("amount cannot be negative")
    if status not in {"NEW", "PAID", "CANCELLED"}:
        errors.append("invalid status")

    return errors

print(validate_order({
    "id": "O1",
    "region": "South",
    "amount": 100,
    "status": "PAID",
}))
```

EX-080 - Even squares

Reference solution

```
even_squares = [
    value ** 2
    for value in range(1, 21)
    if value % 2 == 0
]
print(even_squares)
```

Solutions 081-090

EX-081 - Normalized unique skills

The set removes normalized duplicates. The filter prevents an empty normalized value from entering the result.

Reference solution

```
raw_skills = [" Python ", "SQL", "python", "", " Excel "]
skills = {
    skill.strip().casefold()
    for skill in raw_skills
    if skill.strip()
}
print(skills)
```

EX-082 - Product revenue map

Reference solution

```
rows = [
    ("P1", 5, 100),
    ("P2", 3, 80),
    ("P3", -1, 50),
]
revenue_by_product = {
    product_id: quantity * price
    for product_id, quantity, price in rows
    if quantity >= 0 and price >= 0
}
print(revenue_by_product)
```

EX-083 - Flatten valid readings

Reference solution

```
batches = [[10, None, 12], [8, -1], [15, 16]]
valid = [
    value
    for batch in batches
    for value in batch
    if value is not None and value >= 0
]
print(valid)
```

EX-084 - Record transformation comprehension

The expression is still readable because the transformation and filter are short. If net calculation became more complex, a loop or helper function would be preferable.

Reference solution

```
transactions = [
    {"id": "T1", "status": "SUCCESS", "amount": 100, "fee": 2},
    {"id": "T2", "status": "FAILED", "amount": 80, "fee": 2},
    {"id": "T3", "status": "SUCCESS", "amount": 1, "fee": 2},
]

result = [
    {
        "id": transaction["id"],
        "net_amount": transaction["amount"] - transaction["fee"],
    }
    for transaction in transactions
    if transaction["status"] == "SUCCESS"
    and transaction["amount"] - transaction["fee"] > 0
]
print(result)
```

EX-085 - Status frequency with Counter

Reference solution

```
from collections import Counter

raw = [" ok ", "FAILED", "Ok", "", "pending", "OK"]
normalized = [
    status.strip().upper()
    for status in raw
    if status.strip()
]
counts = Counter(normalized)
print(counts)
print(counts.most_common(1))
```

EX-086 - Inventory requirement gap

Counter subtraction keeps only positive counts, which naturally represents unmet requirements.

Reference solution

```
from collections import Counter

required = Counter(["P1", "P1", "P2", "P3"])
available = Counter(["P1", "P2"])
missing = required - available
print(missing)
```

EX-087 - Department grouping

Reference solution

```
from collections import defaultdict

employees = [
    ("Anita", "Analytics"),
    ("Bharat", "Finance"),
    ("Chitra", "Analytics"),
]
groups = defaultdict(list)

for name, department in employees:
    groups[department].append(name)

for names in groups.values():
    names.sort()

print(dict(groups))
```

EX-088 - Bounded moving average

The deque automatically discards the oldest item once maxlen is reached.

Reference solution

```
from collections import deque

values = [10, 20, 30, 40, 50]
window = deque(maxlen=3)

for value in values:
    window.append(value)
    average = sum(window) / len(window)
    print(list(window), average)
```

EX-089 - Readable immutable sale record

Reference solution

```
from collections import namedtuple

Sale = namedtuple(
    "Sale",
    ["product_id", "region", "quantity", "unit_price"],
)
sales = [
    Sale("P1", "South", 5, 100.0),
    Sale("P2", "North", 3, 80.0),
]

total_revenue = sum(
    sale.quantity * sale.unit_price
    for sale in sales
)
print(total_revenue)
```

EX-090 - Balanced brackets

The stack preserves nesting order. Every closing bracket must match the latest unmatched opening bracket.

Reference solution

```
def is_balanced(text):
    opening = {"(", "[", "{"}
    matching = {")": "(", "]": "[", "}": "{"}
    stack = []

    for character in text:
        if character in opening:
            stack.append(character)
        elif character in matching:
            if not stack or stack.pop() != matching[character]:
                return False

    return not stack

print(is_balanced("a * (b + [c - d])"))
print(is_balanced("([)]"))
```

Solutions 091-100

EX-091 - Support-ticket queue

Reference solution

```
from collections import deque

queue = deque(["T1", "T2", "T3"])
processed = []

while queue:
    ticket = queue.popleft()
    processed.append(ticket)
    if ticket == "T2":
        queue.append("T4")

print(processed)
```

EX-092 - Two-sum indexes

Each value is inserted after checking, so the same position cannot be reused as both elements.

Reference solution

```
def two_sum(values, target):
    index_by_value = {}

    for index, value in enumerate(values):
        needed = target - value
        if needed in index_by_value:
            return index_by_value[needed], index
        index_by_value[value] = index

    return None

print(two_sum([2, 7, 11, 15], 9))
```

EX-093 - Efficient overlap

Reference solution

```
first = [1, 2, 2, 3, 4, 5]
second = [2, 4, 4, 6]
second_set = set(second)
seen = set()
overlap = []

for value in first:
    if value in second_set and value not in seen:
        seen.add(value)
        overlap.append(value)

print(overlap)
```

EX-094 - Top-k products

Reference solution

```
import heapq

products = [
    ("P1", 500),
    ("P2", 900),
    ("P3", 300),
    ("P4", 1200),
    ("P5", 750),
]
top_three = heapq.nlargest(3, products, key=lambda item: item[1])
print(top_three)
```

EX-095 - Parse and aggregate transaction lines

Parsing, conversion, business validation, and aggregation are separate steps. Rejected rows are counted rather than silently disappearing.

Reference solution

```
def aggregate_transactions(lines):
    totals = {}
    accepted = 0
    rejected = 0

    for line in lines:
        parts = line.strip().split("|")
        if len(parts) != 4:
            rejected += 1
            continue

        transaction_id, region, amount_text, status = parts
        region = region.strip()
        status = status.strip().upper()

        try:
            amount = float(amount_text)
        except ValueError:
            rejected += 1
            continue

        if not transaction_id or not region or amount < 0 or status != "SUCCESS":
            rejected += 1
            continue

        totals[region] = totals.get(region, 0.0) + amount
        accepted += 1

    return totals, accepted, rejected

lines = [
    "T1|South|100.5|SUCCESS",
    "T2|North|bad|SUCCESS",
    "T3|South|50|success",
    "broken|row",
]
print(aggregate_transactions(lines))
```

EX-096 - Customer-order summary

The output is initialized from the customer dimension, which guarantees customers with no orders remain present.

Reference solution

```
def customer_order_summary(customers, orders):
    summary = {
        customer["id"]: {
            "name": customer["name"],
            "order_count": 0,
            "total_revenue": 0.0,
        }
        for customer in customers
    }
    unmatched = 0

    for order in orders:
        customer_id = order["customer_id"]
        if customer_id not in summary:
            unmatched += 1
            continue
        metrics = summary[customer_id]
        metrics["order_count"] += 1
        metrics["total_revenue"] += order["amount"]

    for metrics in summary.values():
        count = metrics["order_count"]
        metrics["average_order_value"] = (
            None if count == 0 else metrics["total_revenue"] / count
        )

    return summary, unmatched

customers = [
    {"id": "C1", "name": "Anita"},
    {"id": "C2", "name": "Bharat"},
]
orders = [
    {"customer_id": "C1", "amount": 100},
    {"customer_id": "C1", "amount": 50},
    {"customer_id": "C9", "amount": 80},
]
print(customer_order_summary(customers, orders))
```

EX-097 - Duplicate and conflict detector

The dictionary index makes duplicate-ID lookup average O(1). Equality distinguishes exact repeats from conflicting records.

Reference solution

```
def detect_duplicates(transactions):
    first_by_id = {}
    exact_duplicates = []
    conflicts = []

    for transaction in transactions:
        transaction_id = transaction["id"]
        first = first_by_id.get(transaction_id)

        if first is None:
            first_by_id[transaction_id] = transaction
        elif transaction == first:
            exact_duplicates.append(transaction)
        else:
            conflicts.append({
                "first": first,
                "conflicting": transaction,
            })

    return exact_duplicates, conflicts

transactions = [
    {"id": "T1", "customer_id": "C1", "amount": 100},
    {"id": "T1", "customer_id": "C1", "amount": 100},
    {"id": "T2", "customer_id": "C2", "amount": 80},
    {"id": "T2", "customer_id": "C2", "amount": 90},
]
print(detect_duplicates(transactions))
```

EX-098 - Efficient rolling average

The running sum avoids summing the entire window for every position, producing O(n) time.

Reference solution

```
from collections import deque


def rolling_average(values, window_size):
    if window_size <= 0:
        raise ValueError("window_size must be positive")

    window = deque()
    running_sum = 0.0
    averages = []

    for value in values:
        window.append(value)
        running_sum += value

        if len(window) > window_size:
            running_sum -= window.popleft()

        if len(window) == window_size:
            averages.append(running_sum / window_size)

    return averages

print(rolling_average([10, 20, 30, 40, 50], 3))
```

EX-099 - Inventory reconciliation

The union of dictionary-key views supplies complete product coverage. Missing-source records remain distinct from a measured zero quantity.

Reference solution

```
def reconcile_inventory(erp, physical):
    product_ids = erp.keys() | physical.keys()
    result = []

    for product_id in sorted(product_ids):
        erp_qty = erp.get(product_id)
        physical_qty = physical.get(product_id)

        if erp_qty is None or physical_qty is None:
            variance = None
            status = "MISSING_SOURCE"
        else:
            variance = physical_qty - erp_qty
            if variance == 0:
                status = "MATCH"
            elif variance < 0:
                status = "SHORT"
            else:
                status = "EXCESS"

        result.append({
            "product_id": product_id,
            "erp": erp_qty,
            "physical": physical_qty,
            "variance": variance,
            "status": status,
        })

    return result

print(reconcile_inventory(
    {"P1": 10, "P2": 7},
    {"P1": 8, "P3": 4},
))
```

EX-100 - Product performance ranking capstone

The solution validates rows, aggregates sufficient state by product, derives the ratio after aggregation, sorts once, and adds rank during report construction.

Reference solution

```
def rank_products(records):
    summary = {}

    for record in records:
        units = record["units"]
        returned = record["returned_units"]
        if units < 0 or returned < 0 or returned > units:
            raise ValueError(f"invalid units: {record}")

        product = record["product"]
        metrics = summary.setdefault(product, {
            "units": 0,
            "revenue": 0.0,
            "returned_units": 0,
        })
        metrics["units"] += units
        metrics["revenue"] += record["revenue"]
        metrics["returned_units"] += returned

    ordered = sorted(
        summary.items(),
        key=lambda item: item[1]["revenue"],
        reverse=True,
    )

    report = []
    for rank, (product, metrics) in enumerate(ordered, start=1):
        units = metrics["units"]
        return_rate = None if units == 0 else metrics["returned_units"] / units
        report.append({
            "rank": rank,
            "product": product,
            "units": units,
            "revenue": metrics["revenue"],
            "return_rate": return_rate,
            "flag": (
                "HIGH_RETURN"
                if return_rate is not None and return_rate > 0.10
                else "OK"
            ),
        })

    return report

records = [
    {"product": "P1", "units": 10, "revenue": 1000, "returned_units": 1},
    {"product": "P2", "units": 5, "revenue": 800, "returned_units": 1},
    {"product": "P1", "units": 4, "revenue": 450, "returned_units": 1},
]
print(rank_products(records))
```

Appendix B: Python core cheat sheet

Values and conversion

Need	Pattern
Inspect value and type	print(repr(value), type(value).__name__)
Convert text to integer	number = int(text.strip())
Convert text to float	amount = float(text.strip())
Check missing	value is None
Tolerance comparison	math.isclose(a, b, rel_tol=..., abs_tol=...)

Control flow and iteration

```
if condition:
    ...
elif other_condition:
    ...
else:
    ...

for index, item in enumerate(items):
    ...

for left, right in zip(first, second):
    ...

while condition:
    # Update state so the loop can end.
    ...
```

Collection selection

Structure	Use when	Important property
list	Order and duplicates matter; position access is needed	Mutable sequence
tuple	Fixed-position sequence should not change	Immutable sequence
set	Uniqueness and membership dominate	Unique hashable elements
dict	Values are retrieved or grouped by key	Hashable unique keys
deque	Efficient operations are needed at both ends	O(1) append/pop at ends
Counter	Frequency or multiset behavior is central	Dictionary of counts
defaultdict	Missing groups need automatic initial state	Factory creates values

High-value method map

Type	Methods / operations
str	strip, split, join, replace, find, startswith, endswith, casefold
list	append, extend, insert, pop, remove, sort, reverse, copy
set	add, update, discard, union |, intersection &, difference -
dict	get, setdefault, items, keys, values, pop, update, union |
deque	append, appendleft, pop, popleft, rotate

Complexity quick reference

Operation	Average / typical cost
List index or assignment by index	O(1)
List append / final pop	Amortized O(1) / O(1)
List membership, insert front, remove by value	O(n)
Set membership / add / discard	Average O(1)
Dictionary lookup / insert / delete	Average O(1)
Sorting n items	O(n log n)
One full traversal	O(n)
Nested full traversal of n by n	O(n^2)

Debugging checklist

- Reproduce the problem from a clean run.
- Read the final traceback line, then inspect the referenced source line.
- Print repr(value), type(value), and relevant collection lengths.
- Reduce the input to the smallest example that still fails.
- Trace state before and after mutation.
- Verify boundary conditions and empty input.
- Change one thing, rerun, and preserve the failing example as a test.

Appendix C: Glossary

Term	Meaning
Argument	A value supplied when calling a function.
Assignment	Binding a name to an object.
Big O	A notation describing how resource use grows with input size.
Boolean	A value representing True or False.
Comprehension	Compact syntax for constructing a collection from an iterable.
Dictionary	A mutable mapping from unique hashable keys to values.
Expression	Code that evaluates to a value.
Generator	An iterator that produces values lazily.
Hashable	Having stable hash and equality behavior so an object can be a set member or dictionary key.
Identity	Whether two references point to the same object.
Immutable	Unable to change value or structure after creation.
Iterable	An object that can provide items one at a time.
Iterator	An object that returns the next item and remembers traversal state.
Key	The hashable identifier used to retrieve a dictionary value.
Mutation	Changing an existing object in place.
Parameter	A name declared in a function definition.
Projection	Selecting or deriving fields for an output record.
Rebinding	Making a name refer to a different object.
Scope	The region in which a name can be resolved.
Sequence	An ordered collection supporting positional access.
Shallow copy	A new outer container that still references the same nested objects.
Truthiness	The Boolean interpretation of a value in a condition.
Unpacking	Assigning items from an iterable to several target names.

Appendix D: Final readiness assessment

Complete this assessment without notes. A strong result requires correct code, explanation, testing, and complexity reasoning. Time limits are targets, not reasons to skip edge cases.

Section	Task	Target
Syntax and types	Write conversions, formatted output, and explicit missing-state checks	15 minutes
Control flow	Implement a multi-priority rule with boundary tests	15 minutes
Functions	Design a validated function with stable return shape	20 minutes
Lists and tuples	Solve rotation, merge, and fixed-return problems	30 minutes
Sets and dictionaries	Deduplicate, count, group, and join efficiently	35 minutes
Integrated problem	Parse, validate, aggregate, report quality counts, state complexity	45 minutes

Readiness rubric

Level	Evidence
Rebuilding	Can follow examples but needs notes for basic syntax and structure choice
Functional	Solves routine problems but misses some boundaries or complexity implications
Confident	Writes correct, tested solutions and explains data-structure choices
Interview-ready	Solves under time, communicates trade-offs, and handles edge cases deliberately
Expert foundation	Produces clean reusable functions, audits data quality, and optimizes only where justified

COMPLETION STANDARD

You are ready for the next Python-to-data-analytics volume when you can solve at least 80 of the 100 exercises from blank files, score Confident or higher on every rubric section, and explain list, set, dictionary, tuple, deque, and Counter trade-offs without notes.